

Belief Propagation for Min-Cost Network Flow: Convergence and Correctness

David Gamarnik

Operations Research Center and Sloan School of Management, Massachusetts Institute of Technology,
Cambridge, Massachusetts 02139, gamarnik@mit.edu

Devavrat Shah

Laboratory for Information and Decision Systems (LIDS) and Operations Research Center, Department of EECS,
Massachusetts Institute of Technology, Cambridge, Massachusetts 02139, devavrat@mit.edu

Yehua Wei

Operations Research Center, Massachusetts Institute of Technology, Cambridge, Massachusetts 02139,
y4wei@mit.edu

Distributed, iterative algorithms operating with minimal data structure while performing little computation per iteration are popularly known as *message passing* in the recent literature. Belief propagation (BP), a prototypical message-passing algorithm, has gained a lot of attention across disciplines, including communications, statistics, signal processing, and machine learning as an attractive, scalable, general-purpose heuristic for a wide class of optimization and statistical inference problems. Despite its empirical success, the theoretical understanding of BP is far from complete.

With the goal of advancing the state of art of our understanding of BP, we study the performance of BP in the context of the capacitated minimum-cost network flow problem—a cornerstone in the development of the theory of polynomial-time algorithms for optimization problems and widely used in the practice of operations research. As the main result of this paper, we prove that BP converges to the optimal solution in pseudopolynomial time, provided that the optimal solution of the underlying network flow problem instance is unique and the problem parameters are integral. We further provide a simple modification of the BP to obtain a fully polynomial-time randomized approximation scheme (FPRAS) without requiring uniqueness of the optimal solution. This is the first instance where BP is proved to have fully polynomial running time. Our results thus provide a theoretical justification for the viability of BP as an attractive method to solve an important class of optimization problems.

Subject classifications: belief propagation; network flow; graphical model.

Area of review: Optimization.

History: Received December 2009; revisions received August 2010, June 2011; accepted August 2011.

1. Introduction

Message passing has emerged as canonical algorithmic architecture to deal with the scale of the optimization and inference problems arising in the context of variety of disciplines, including communications, networks, machine learning, image processing, and computer vision, signal processing, and statistics. The Belief Propagation (BP) is a message-passing heuristic for solving optimization and inference problems in the context of the graphical model. The graphical model or a Markov random field provides a succinct representation for capturing the dependency structure between a collection of random variables. In recent years, the need for large-scale statistical inference and optimization has made graphical models the representation of choice in a variety of applications. There are two key problems for a graphical model of interest. The first problem is the computation of marginal distribution of a random variable. This problem is (computationally) equivalent to the computation of the so-called partition function and can be

thought of as a weighted combinatorial counting problem (e.g., counting the number of independent sets of a graph is a special case of this problem). The second problem is that of finding the mode of a distribution, i.e., an assignment with the maximum likelihood (ML). For a constrained optimization (maximization) problem, when the constraints are modeled through a graphical model and probability is proportional to the cost of the assignment, an ML assignment is an optimal solution to the optimization problem. Both of these questions, in general, are computationally hard either in the #P or NP-complete sense.

Belief Propagation (BP) is an “umbrella” message-passing heuristic designed for these two problems. Its version for the first problem is known as the “sum-product algorithm” and for the second problem is known as the “max-product” or “min-sum algorithm.” Both versions of the BP algorithm are iterative, easy to implement, and distributed in nature. When the underlying graph is a tree, the BP algorithm essentially performs the dynamic programming recursion

(Gallager 1963, Yedidia et al. 2002, Pearl 1988) and, as a result, leads to a correct solution both for the optimization and inference problems. Specifically, BP provides a natural parallel iterative version of the dynamic programming in which variable nodes pass messages between each other along edges of the graphical model. Somewhat surprisingly, this seemingly naive BP heuristic has become quite popular in practice, even for graphical models that do not have the tree structure (Aji and McEliece 2000, Horn 1999, Mezard et al. 2002, Richardson and Urbanke 2001). In our opinion, there are two primary reasons for the popularity of BP. First, it is generically applicable, easy to understand, and implementation friendly due to its iterative, simple, and message-passing nature. Second, in many practical scenarios, the performance of BP is surprisingly good (Weiss and Freeman 2001, Yedidia et al. 2002). On one hand, for an optimist this unexpected success of BP provides a hope for it being a genuinely much more powerful algorithm than what we know thus far (e.g., better than primal-dual methods). On the other hand, a skeptic would demand a systematic understanding of the limitations (and strengths) of BP in order to caution a practitioner. Thus, irrespective of the perspective of an algorithmic theorist, rigorous understanding of BP is very important.

Despite the apparent empirical success of the BP algorithm for solving a variety of problems, theoretical understanding of BP is far from complete. In this paper, our interest lies primarily in the correctness and convergence properties of the min-sum version of BP when applied to the minimum-cost network flow problems (or simply min-cost flow)—an important class of linear (or more generally convex) optimization problems. As a secondary interest, we wish to bring BP to the attention of researchers in the operations research (OR) community, thereby improving the current state in which BP has remained elusive in OR.

1.1. Contributions

As the main contribution of this paper, we establish that BP converges to the optimal solution of a min-cost network flow problem in pseudopolynomial time, provided that the optimal solution of the underlying problem is unique and the problem input is integral. At the same time, it is known (Sanghavi et al. 2009) that BP fails to converge for the general linear programming (LP) problem by means of a counterexample. Thus, our results extend, in an important way, the scope of the problems that are provably solvable by the BP algorithm. We also point out that identifying the broadest class of optimization problems solvable using the BP algorithm is an interesting open problem. Indeed, resolution of it will lead to the precise understanding of the structure of optimization problems that are solvable by BP.

The contributions of this paper, in detail, are as follows. First, we show that an exact version of BP can be implemented for the min-cost flow problems by encoding each message in BP as a piecewise-linear convex function. This is significant because the natural formulation of BP

requires maintaining a vector of real-valued functions that may require an infinite amount of memory to store and computation to update. Then we provide a proof to show that BP finds the optimal solution in pseudopolynomial time, provided that the optimal solution is unique. Next, we present a simple modification of the BP algorithm that gives a fully polynomial-time randomized approximation scheme (FPRAS) for the same problem, which no longer requires the uniqueness of the optimal solution. This is the first instance where BP is proved to have fully polynomial running time, except for the case when the underlying graph is a tree and BP solves the problem exactly. The modification of BP is obtained by applying a novel lemma; it is a natural generalization of the so-called Isolation Lemma found in Mulmuley et al. (1987). Unlike the Isolation Lemma, our lemma can be used for generic LP. In essence, we show that it is possible to perturb the cost of any LP using *little* randomness so that the resulting modified LP has a unique solution that is a good approximation to the original LP, and its gap to the next optimal solution is *large enough*. Indeed, this is a general method and can be useful in a variety of applications, including improving performance of distributed algorithms; it is no surprise that it has already been used in a subsequent work (Kanoria et al. 2011).

1.2. Prior Work on BP

Despite the compelling reasons explained earlier, only recently we have witnessed an explosion of research for theoretical understanding of the performance of the BP algorithm in the context of various combinatorial optimization problems, both tractable and intractable (NP-hard) versions. In earlier work, Weiss and Freeman (2001) identified certain local optimality properties of the BP (max-product) for arbitrary graphs. It implies that when a graph has a single cycle, then the fixed point of max-product corresponds to the correct answer. However, they do not provide any guarantee on the convergence of max-product. Bayati et al. (2008a) considered the performance of BP for finding the maximum weight matching in a bipartite graph. They established that BP converges in pseudo polynomial time to the optimal solution when the optimal solution is unique (Bayati et al. 2008a). Bayati et al. (2008b), as well as Sanghavi et al. (2007), generalized this result by establishing the correctness and convergence of the BP algorithm for the b -matching problem when the linear programming relaxation corresponding to the node constraints has a unique integral optimal solution. Note that the LP relaxation corresponding to the node constraints is not tight in general, because inclusion of the odd-cycle elimination constraints (Schrijver 2003) is essential. Furthermore, Bayati et al. (2008b) and Sanghavi et al. (2007) established that the BP does not converge if this LP relaxation does have a nonintegral solution. Thus, for a b -matching problem, BP finds an optimal answer when the LP relaxation can find an optimal solution. In the context of the maximum weight independent set problem, a one-sided relation between LP relaxation and

BP is established (Sanghavi et al. 2009); if BP converges, then it is correct, and LP relaxation is tight. In Sanghavi et al. (2009), a counterexample was produced that shows that BP does not converge to the optimal solution of an LP. This seems to suggest that BP is unlikely to solve all forms of LP.

Beyond LP, the performance of BP for quadratic optimization problems (QP) and, more generally, convex optimization problems (CP) have recently been studied. The conditions for correctness and convergence of BP in the context of inference in Gaussian graphical models such as those established by Malioutov et al. (2006) lead to sufficient conditions for when BP can solve (a certain class of) QP. More recently, in a sequence of works, Moallemi and Van Roy (2007, 2009) have identified sufficient conditions under which BP converges to correct solution for convex optimization problems. It is worth identifying the differences between results of this paper and that of Moallemi and Van Roy (2007, 2009). To start with, our work applies to the constrained min-cost network flow LP, whereas that of Moallemi and Van Roy (2007, 2009) applies to the unconstrained convex optimization problem. Whereas the constrained min-cost network flow LP can be seen as an unconstrained convex optimization problem (e.g., via Lagrangian relaxation), the resulting convex optimization is not *strictly* convex, and hence sufficient conditions (the diagonal dominance of Hessian) of Moallemi and Van Roy (2007, 2009) are not applicable. Indeed, the proof methods are different, and results of this paper provide “implementation” of BP, unlike the results of Moallemi and Van Roy (2007, 2009). We also take note of a work by Ruozzi and Tatikonda (2008) that utilizes BP to find source-sink paths in the network.

1.3. Prior Work on Min-Cost Network Flow

The min-cost network flow problem ($\mathcal{MCF}$) has been fundamental in the development of the theory of polynomial-time algorithms for optimization problems. The first polynomial-time algorithm for $\mathcal{MCF}$ was developed by Edmonds and Karp (1972) with a running time of $O(m(\log U)(m + n \log n))$, where m represents the number of edges, n represents the number of nodes, and U the largest capacity of an arc. Subsequently, the first strongly polynomial-time algorithm was proposed by Tardos (1985). Because $\mathcal{MCF}$ has been central to the development of algorithmic theory, a wide variety of efficient algorithms have been proposed over years with different virtues such as Röck (1980), Orlin (1993), Fujishige (1986), Bertsekas (1986), Goldberg and Tarjan (1987, 1989), Ahuja et al. (1992). Among these, the fastest polynomial-time algorithm runs (evaluated in the centralized computation model) in essentially $O(n^3 \log(nC))$ time (Bertsekas 1986, Goldberg and Tarjan 1989, Ahuja et al. 1992), where C is the largest cost of an arc. On the other hand, the fastest strongly polynomial-time algorithm for $\mathcal{MCF}$ runs (again, evaluated in the centralized computation model) in $O(m \log n(m + n \log n))$ (Orlin 1993).

It is worth comparing the running time of the BP algorithm that we have obtained for $\mathcal{MCF}$. The basic version of BP takes (evaluated under decentralized computation model) $O(C^3 mn^4 \log n)$ computation (C represents the largest cost) in total. The modified FPRAS version of BP algorithm requires $O(\varepsilon^{-3} n^8 m^7 \log n)$ computation in total on average (w.r.t. decentralized computation model) for obtaining $(1 + \varepsilon)$ approximation. It should be noted that the number of iterations required by the algorithm scales as nL where L is the maximal cost of a directed path.

It is clear from the comparison that the bounds implied by our results for BP are not competitive with respect to the best known results for $\mathcal{MCF}$. BP's performance is evaluated for the decentralized model, whereas the above reported computation time analysis for other algorithms is for the centralized model. Indeed, some of the known algorithms can be implemented in the decentralized model, such as that of Bertsekas (1986) and Goldberg and Tarjan (1987) (see Ahuja et al. 1993, Chapters 10–12 for further details). The analysis of BP for $\mathcal{MCF}$, when specialized to specific instances of $\mathcal{MCF}$ like the bipartite matching problem, leads to tighter performance bounds that are competitive with respect to the best known results (see §4.2, Theorem 4.14). However, the important thing is that BP is a general-purpose algorithm, not specialized for the problem at hand like the best known algorithm for $\mathcal{MCF}$. For this reason, BP is highly desirable from an implementor's perspective because it does not require specific modifications for the problem of interest. Finally, it should be noted that the BP algorithm can operate in an asynchronous decentralized environment, unlike most known algorithms.

1.4. Organization

The rest of the paper is organized as follows. In §2, we introduce the BP algorithm as an iterative heuristic for a generic optimization problem. We provide an intuitive explanation by means of an example of how BP is derived as an iterative heuristic for the generic problem inspired by parallel implementation of dynamic programming on a tree-like problem structure. In §3, we specialize the BP for linear programming (LP). We recall a (counter-)example of an LP for which BP cannot find its optimal solution. In §4, we further specialize the BP algorithm for the capacitated min-cost network flow problem ($\mathcal{MCF}$). We state the main result that establishes pseudopolynomial time convergence of BP to the optimal solution of $\mathcal{MCF}$, when the optimal solution is unique. Specifically, §4.1 explains how each message function in the BP algorithm can be computed, leading to an efficient implementation of BP. In §4.2, we consider a subclass $\mathcal{MCF}^o$ of $\mathcal{MCF}$ that includes the problems of min-cost path, as well as bipartite matching or, more generally, b -matching. For this subclass of $\mathcal{MCF}$, it turns out that BP has very simple message functions, and this subsequently leads to a tighter bound on the running time. In §5, the proof of the main result about convergence of BP for $\mathcal{MCF}$ is provided. Section 6 presents an extension of our result for

min-cost flow problems with piecewise-linear convex cost functions. In §7, we provide the running time analysis of BP for $\mathcal{MCF}$ and $\mathcal{MCF}^o$. From the analysis, we show that BP for the min-cost flow problem is a pseudopolynomial-time algorithm when the data input is integral. In §8, we present a randomized approximation scheme for the min-cost flow problem that uses the standard BP as a subroutine. We prove that for any $\varepsilon \in (0, 1)$, the approximation scheme finds a solution that is within $1 + \varepsilon$ of the optimal solution, whereas its expected running time is polynomial in m , n , and $1/\varepsilon$. In doing so, we introduce a variation of the Isolation Lemma for LP in §8.1. Finally, §9 presents conclusions and directions for future work.

2. Belief Propagation for Optimization Problem

Here we introduce the min-sum version of BP as a heuristic for the optimization problem in the general form. We shall utilize the notations similar to those used in Moallemi and Van Roy (2007, 2009). In the remainder of the paper, by BP we mean its min-sum version for solving optimization problem. To this end, consider the optimization problem

$$\text{minimize } \sum_{i \in V} \phi_i(x_i) + \sum_{C \in \mathcal{C}} \psi_C(x_C) \quad (\mathcal{P})$$

$$\text{subject to } x_i \in \mathbb{R}, \quad \forall i \in V,$$

where V is a finite set of *variables* and $\mathcal{C}$ is a finite collection of subsets of V representing *constraints*. Here $\phi_i: \mathbb{R} \rightarrow \bar{\mathbb{R}}, \forall i \in V$ and $\psi_C: \mathbb{R}^{|C|} \rightarrow \bar{\mathbb{R}}, \forall C \in \mathcal{C}$ are extended real-valued functions where $\bar{\mathbb{R}}$ represents extended real numbers $\mathbb{R} \cup \{\infty\}$. We call each ϕ_i a *variable function*, each ψ_C a *factor function*, and $(\mathcal{P})$ a *factorized optimization problem*.

It is not difficult to see that essentially any constrained optimization problem of interest can be represented as a factorized optimization problem. For example, consider the well-known maximum-size independent set problem on a simple undirected graph $G = (V, E)$, which requires selecting subset V of maximal cardinality so that no two vertices of the chosen subset are neighbors of each other as per E . The factorized form of the maximum weight independent set is given by

$$\text{minimize } \sum_{i \in V} \phi_i(x_i) + \sum_{(i, j) \in E} \psi_{ij}(x_i, x_j)$$

$$\text{subject to } x_i \in \mathbb{R}, \quad \forall i \in V,$$

where

$$\phi_i(x_i) = \begin{cases} 0 & \text{if } x_i = 0 \\ -1 & \text{if } x_i = 1 \\ \infty & \text{otherwise,} \end{cases}$$

$$\psi_{ij}(x_i, x_j) = \begin{cases} 0 & \text{if } x_i + x_j \leq 1 \\ \infty & \text{otherwise.} \end{cases}$$

In the above, $x_i = 1$ if and only if node i is selected in the independent set. Finally, we introduce the notion of a factor graph of a factorized optimization problem. A factor graph $F_{\mathcal{P}}$ of $(\mathcal{P})$ is a bipartite graph with one partition containing *variable* nodes V and the other partition containing *factor* nodes $\mathcal{C}$ corresponding to the constraints. There is an edge $(v, C) \in V \times \mathcal{C}$ if and only if $v \in C$. For example, the graph shown in Figure 1 is the factor graph for the optimization problem

$$\begin{aligned} \text{minimize } & \left(\sum_{i=1}^5 \phi_i(x_i) \right) + \psi_{1,2,3}(x_1, x_2, x_3) \\ & + \psi_{1,4,5}(x_1, x_4, x_5) + \psi_{1,5}(x_1, x_5) \end{aligned} \quad (\mathcal{P}')$$

$$\text{subject to } x_i \in \mathbb{R}, \quad \forall 1 \leq i \leq 5.$$

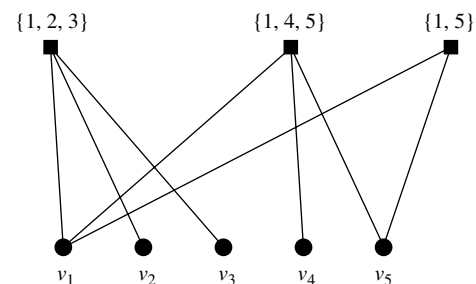
Now we introduce BP. To start with, suppose the factor graph $F_{\mathcal{P}}$ of $\mathcal{P}$ is a tree (note that the factor graph in Figure 1 is *not* a tree because there is a cycle $(v_1, \{1, 4, 5\}, v_5, \{1, 5\}, v_1)$). In this case, let us consider the dynamic programming algorithm. The dynamic programming algorithm would suggest computation of the value or assignment of a given variable node $i \in V$ in the optimal solution as follows: fix a specific value $z \in \mathbb{R}$ of variable x_i corresponding to the variable $i \in V$. Subject to $x_i = z$, compute the cost of optimal assignment for the rest of the problem, say $b_i(z)$. Then the optimal assignment of variable node i is in $\arg \min_{z \in \mathbb{R}} b_i(z)$. Now to compute $b_i(z)$ for all $z \in \mathbb{R}$, the dynamic programming would recurse the same approach on the problem

$$\text{minimize } \phi_i(z) + \sum_{j \in V \setminus \{i\}} \phi_j(x_j) + \sum_{C \in \mathcal{C}} \psi_C(x_C), \quad (1)$$

$$\text{subject to } x_i = z, x_j \in \mathbb{R}, \quad \forall j.$$

Now, implementation of this recursion of dynamic programming is generally not straightforward and can be computationally expensive. However, when the factor graph $F_{\mathcal{P}}$ is a tree, it is quite simple because the problem decomposes into subproblems on disconnected trees. It is the dynamic programming implementation for the tree factor graph that leads to the derivation of BP. To that end, given a node i consider any constraint C such that $i \in C$, i.e., (i, C) is an edge in $F_{\mathcal{P}}$. Because $F_{\mathcal{P}}$ is a tree, $F_{\mathcal{P}} \setminus (i, C)$ has two disjoint components, say T_1 and T_2 . Without loss of generality,

Figure 1. An example of a factor graph.



we assume that i is contained in T_1 and C is contained in T_2 . Due to this division of the problem structure, $b_i(z)$ for $z \in \mathbb{R}$ or, equivalently, solution of optimization problem (1), can be computed recursively as follows. For edge (i, C) , define “messages” $m_{i \rightarrow C}(z)$ and $m_{C \rightarrow i}(z)$ as

$$\begin{aligned} m_{i \rightarrow C}(z) = & \text{minimize} \sum_{j \in V \cap T_1} \phi_j(x_j) + \sum_{D \in \mathcal{C} \cap T_1} \psi_D(x_D), \\ & \text{subject to } x_i = z, x_j \in \mathbb{R}, \quad \forall j. \\ m_{C \rightarrow i}(z) = & \text{minimize} \sum_{j \in V \cap T_2} \phi_j(x_j) + \sum_{D \in \mathcal{C} \cap T_2} \psi_D(x_D), \\ & \text{subject to } x_i = z, x_j \in \mathbb{R}, \quad \forall j. \end{aligned}$$

Note that two such directional “messages” can be defined for any edge in $F_{\mathcal{P}}$ in a similar manner because it is a tree. Again, invoking the tree structure of $F_{\mathcal{P}}$ and definition of “messages,” the solution of (1) can be rewritten as

$$b_i(z) = \phi_i(z) + \sum_{C \in \mathcal{C}_i} m_{C \rightarrow i}(z), \quad \forall z \in \mathbb{R}, \quad (2)$$

where $\mathcal{C}_i$ is the set of all factor nodes (or constraints) that contain i , i.e.,

$$\mathcal{C}_i \triangleq \{C \in \mathcal{C} : i \in C\}.$$

That is, if the graph underlying $F_{\mathcal{P}}$ is a tree, then in order to compute $b_i(z)$ it is sufficient to have knowledge of the “messages” coming towards node i from the factor nodes to which it is connected. For the tree $F_{\mathcal{P}}$, such messages can be recursively defined as follows: for any edge (i, C) in $F_{\mathcal{P}}$, for any $z \in \mathbb{R}$,

$$m_{i \rightarrow C}(z) = \phi_i(z) + \sum_{K \in \mathcal{C}_i \setminus C} m_{K \rightarrow i}(z), \quad (3)$$

$$m_{C \rightarrow i}(z) = \min_{\substack{y \in \mathbb{R}^{|C|} \\ y_i = z}} \psi_C(y) + \sum_{j \in C \setminus i} m_{j \rightarrow C}(y_j). \quad (4)$$

For tree-structured $F_{\mathcal{P}}$, starting from leaf nodes using (3)–(4) the “messages” $m_{i \rightarrow C}(z)$ and $m_{C \rightarrow i}(z)$ for all edges (i, C) can be computed. A parallel implementation of this recursive procedure is as follows. Initially, for $t = 0$ we set $m_{C \rightarrow i}^0(z) = m_{i \rightarrow C}^0(z) = 0$ for all edges (i, C) of $F_{\mathcal{P}}$. For $t \geq 1$, update messages for each edge (i, C) of $F_{\mathcal{P}}$ as

$$m_{i \rightarrow C}^t(z) = \phi_i(z) + \sum_{K \in \mathcal{C}_i \setminus C} m_{K \rightarrow i}^{t-1}(z), \quad (5)$$

$$m_{C \rightarrow i}^t(z) = \min_{\substack{y \in \mathbb{R}^{|C|} \\ y_i = z}} \psi_C(y) + \sum_{j \in C \setminus i} m_{j \rightarrow C}^t(y_j). \quad (6)$$

The estimation of $b_i(z)$ at the end of iteration t for each $i \in V$ and $z \in \mathbb{R}$ is given by

$$b_i^t(z) = \phi_i(z) + \sum_{C \in \mathcal{C}_i} m_{C \rightarrow i}^t(z). \quad (7)$$

It is easy to show by induction that if the graph underlying $F_{\mathcal{P}}$ is a tree, then for t larger than the diameter of the tree, $b_i^t(\cdot)$ equals the value produced by the dynamic programming problem, therefore resulting in the optimal assignment of x_i .

The parallelized implementation of the dynamic programming problem described by (5) and (6) can be applied to any factor graph in general. This is precisely the BP min-sum heuristic. The algorithm is described in detail next. For the nontree graphs the convergence and/or correctness of such a heuristic is by no means guaranteed in general.

Algorithm 1 min-sum BP

- 1: Given a factorized optimization problem $(\mathcal{P})$,
construct factor graph $F_{\mathcal{P}}$.
- 2: Set N to be the number of iterations for BP.
- 3: Initialize $t = 0$, and for each edge (i, C) in $F_{\mathcal{P}}$,
initialize $m_{C \rightarrow i}^0(z) = 0 = m_{i \rightarrow C}^0(z)$ for all $z \in \mathbb{R}$.
- 4: **for** $t = 1, 2, \dots, N$ **do**
- 5: For any edge (i, C) in $F_{\mathcal{P}}$ and $z \in \mathbb{R}$, update

$$m_{i \rightarrow C}^t(z) = \phi_i(z) + \sum_{K \in \mathcal{C}_i \setminus C} m_{K \rightarrow i}^{t-1}(z), \quad (8)$$

$$m_{C \rightarrow i}^t(z) = \min_{\substack{y \in \mathbb{R}^{|C|} \\ y_i = z}} \psi_C(y) + \sum_{j \in C \setminus i} m_{j \rightarrow C}^t(y_j). \quad (9)$$

- 6: $t := t + 1$
- 7: **end for**
- 8: Set the belief function as
 $b_i^N(z) = \phi_i(z) + \sum_{C \in \mathcal{C}_i} m_{C \rightarrow i}^N(z)$, $\forall 1 \leq i \leq n$.
- 9: Estimate the optimal assignment as
 $\hat{x}_i^N \in \arg \min b_i^N(z)$ for each $i \in V$.
- 10: Return $\hat{x}^N$.

3. BP for Linear Programming

The linear programming (LP) problem in the standard form is given by

$$\begin{aligned} & \text{minimize} \quad c^T x \\ & \text{subject to} \quad Ax = g, \\ & \quad \quad \quad x \geq 0, x \in \mathbb{R}^n, \end{aligned} \quad (\mathcal{LP})$$

where $A \in \mathbb{R}^{m \times n}$, $g \in \mathbb{R}^m$, and $c \in \mathbb{R}^n$. In the notation of the factorized optimization problem introduced earlier, variable nodes are $V = \{1, \dots, n\}$ with associated variables x_i , $i \in V$; rows of A correspond to constraint nodes $\mathcal{C} = \{C_j : 1 \leq j \leq m\}$ where $C_j = \{i \in V : a_{ji} \neq 0\}$; and $\mathcal{C}_i = \{C_j : a_{ji} \neq 0\}$, $\forall i \in V$. Define factor function $\psi_j : \mathbb{R}^{|C_j|} \rightarrow \bar{\mathbb{R}}$ for $1 \leq j \leq m$ as

$$\psi_j(z) = \begin{cases} 0 & \text{if } \sum_{i \in C_j} a_{ji} z_i = g_j \\ \infty & \text{otherwise.} \end{cases}$$

And define variable function $\phi_i: \mathbb{R} \rightarrow \bar{\mathbb{R}}$ for $i \in V$ as

$$\phi_i(z) = \begin{cases} c_i z & \text{if } z \geq 0 \\ \infty & \text{otherwise.} \end{cases}$$

Then, $(\mathcal{LP})$ is equivalent to following the factorized optimization problem:

$$\begin{aligned} & \text{minimize} \quad \sum_{i=1}^n \phi_i(x_i) + \sum_{j=1}^m \psi_{C_j}(x_{C_j}), \\ & \text{subject to} \quad x_i \in \mathbb{R}, \quad \forall i \in V. \end{aligned} \quad (\mathcal{P}_{\mathcal{LP}})$$

Then BP for this factorized optimization problem becomes the BP heuristic for LP. Note that the BP described earlier requires computing message functions of the form $m_{i \rightarrow C}^t$ and $m_{C \rightarrow i}^t$. In general, it is not clear if such message functions can be stored and updated efficiently. For LP, however, it can be shown that every message function is a piecewise-linear convex function, which allows efficient encoding of them in terms of a finite vector describing the break points and the slopes of its linear pieces. In §4.1, we will do this in the context of the min-cost network flow problem, and we will explain the associated computation procedure in detail.

Now, BP being a distributed algorithm, it is unlikely to work well when the $(\mathcal{LP})$ does not have a unique optimal solution. However, even with the assumption that $(\mathcal{LP})$ has a unique optimal solution, in general the estimation of BP may not converge to the unique optimal solution. One such instance is an LP-relaxation of the maximum weight independent set problem on a complete bipartite graph (Sanghavi et al. 2009):

$$\begin{aligned} & \text{minimize} \quad -\sum_{i=1}^3 2x_i - \sum_{j=1}^3 3y_j \\ & \text{subject to} \quad x_i + y_j + z_{ij} = 1, \quad \forall 1 \leq i, j \leq 3, \\ & \quad \quad \quad x, y, z \geq 0. \end{aligned} \quad (\mathcal{P}_J)$$

Although BP in Sanghavi et al. (2009) was stated in a somewhat different manner, it can be checked that it is equivalent to the description presented here. It turns out that although this problem has a unique optimal solution, the BP algorithm does not converge at all, let alone to the optimal solution. Specifically, the messages $\hat{x}^N$ oscillate between two different values as the number of iterations N oscillates between odd and even values.

4. BP Algorithm for Min-Cost Network Flow Problem

In this section, we formulate BP for the capacitated min-cost network flow problem $(\mathcal{MCF})$ and state our main result about the convergence of BP for $\mathcal{MCF}$. As mentioned earlier, each message of BP for $\mathcal{MCF}$ is a function, and we describe how these messages can be efficiently updated and stored as vectors in §4.1. In §4.2, we consider a subclass

of $\mathcal{MCF}$; it includes bipartite matching, for which BP can take advantage of its special structure to obtain much faster running time.

Let us define the capacitated min-cost network flow problem $(\mathcal{MCF})$. Given a directed graph $G = (V, E)$, let V, E denote the set of vertices and arcs or directed edges, respectively, with $|V| = n$ and $|E| = m$. For any vertex $v \in V$, let E_v be the set of arcs incident to v , and for any $e \in E_v$, let $\Delta(v, e) = 1$ if e is an *out-arc* of v (i.e., arc $e = (v, w)$, for some $w \in V$), and $\Delta(v, e) = -1$ if e is an *in-arc* of v (i.e., arc $e = (w, v)$, for some $w \in V$). The $\mathcal{MCF}$ on G is formulated as follows (Ahuja et al. 1993, Bertsimas and Tsitsiklis 1997):

$$\begin{aligned} & \text{minimize} \quad \sum_{e \in E} c_e x_e \\ & \text{subject to} \quad \sum_{e \in E_v} \Delta(v, e) x_e = f_v, \quad \forall v \in V \\ & \quad \quad \quad \text{(demand/supply constraints)} \\ & \quad \quad \quad 0 \leq x_e \leq u_e, \quad \forall e \in E \text{ (flow constraints)} \end{aligned} \quad (\mathcal{MCF})$$

where $c_e \geq 0$, $u_e \geq 0$, $c_e \in \mathbb{R}$, $u_e \in \bar{\mathbb{R}}$, for each $e \in E$, and $f_v \in \mathbb{R}$ for each $v \in V$. The variables x_e represent flow value assigned to each arc $e \in E$; the first type of constraints state that the difference of inflow and outflow at each node $v \in V$ equals the node demand f_v (could be positive or negative), and the second type of constraints state that flow on each arc $e \in E$ is nonnegative and cannot be larger than its capacity u_e . We shall assume that the instance of network flow is feasible. Without loss of generality, let each node $v \in V$ be such that $|E_v| \geq 2$, or else either $E_v = \emptyset$, in which case we ignore such v , or $|E_v| = 1$, in which case the flow on $e \in E_v$ is determined by f_v . For the $\mathcal{MCF}$, define factor and variable functions ψ, ϕ as follows: for $v \in V, e \in E$

$$\begin{aligned} \psi_v(z) &= \begin{cases} 0 & \text{if } \sum_{e \in E_v} \Delta(v, e) z_e = f_v, \\ \infty & \text{otherwise,} \end{cases} \\ \phi_e(z) &= \begin{cases} c_e z & \text{if } 0 \leq z \leq u_e, \\ \infty & \text{otherwise.} \end{cases} \end{aligned}$$

Then, solving $\mathcal{MCF}$ is equivalent to solving $\min_{x \in \mathbb{R}^{|E|}} \{ \sum_{v \in V} \psi_v(x_{E_v}) + \sum_{e \in E} \phi_e(x_e) \}$. Therefore, the BP algorithm can be applied for $\mathcal{MCF}$ in this standard form. Because of the special structure of $\mathcal{MCF}$ that each variable node is adjacent to exactly two factor nodes, it is indeed possible to skip the message update step $m_{v \rightarrow e}^t$, resulting in a simplified Algorithm 2, stated next.

Algorithm 2 BP for $\mathcal{MCF}$

- 1: Initialize $t = 0$, messages $m_{e \rightarrow v}^0(z) = 0$, $m_{e \rightarrow w}^0(z) = 0$, $\forall z \in \mathbb{R}$ for each $e = (v, w) \in E$.
- 2: **for** $t = 1, 2, 3, \dots, N$ **do**
- 3: For each $e = (v, w) \in E$, update messages as follows:

$$m_{e \rightarrow v}^t(z) = \phi_e(z) + \min_{\bar{z} \in \mathbb{R}^{|E_w|}, \bar{z}_e = z} \left\{ \psi_w(\bar{z}) + \sum_{\bar{e} \in E_w \setminus e} m_{\bar{e} \rightarrow w}^{t-1}(\bar{z}_{\bar{e}}) \right\},$$

$$\forall z \in \mathbb{R}$$

$$m_{e \rightarrow w}^t(z) = \phi_e(z) + \min_{\bar{z} \in \mathbb{R}^{|E_v|}, \bar{z}_e = z} \left\{ \psi_v(\bar{z}) + \sum_{\bar{e} \in E_v \setminus e} m_{\bar{e} \rightarrow v}^{t-1}(\bar{z}_{\bar{e}}) \right\},$$

$$\forall z \in \mathbb{R}$$

4: $t := t + 1$

5: **end for**

6: For each $e = (v, w) \in E$, set the belief function as

$$b_e^N(z) = \phi_e(z) + \sum_{\bar{e} \in E_v \setminus e} m_{\bar{e} \rightarrow v}^{N-1}(z) + \sum_{\bar{e} \in E_w \setminus e} m_{\bar{e} \rightarrow w}^{N-1}(z).$$

7: Calculate the belief estimate by finding

$$\hat{x}_e^N \in \arg \min b_e^N(z) \text{ for each } e \in E.$$

8: Return $\hat{x}^N$ as an estimation of the optimal solution of $\mathcal{MCF}$.

Intuitively, in Algorithm 2 each arc can be thought of as an agent who is trying to figure out its own flow while meeting the conservation constraints at its endpoints. Each link maintains an estimate of its “local cost” as a function of its flow (thus, this estimate is a function, not a single number). At each time step an arc updates its function as follows: the cost of assigning x units of flow to link e is the cost of pushing x units of flow through e plus the minimum-cost way of assigning flow to neighboring edges (with respect to the functions computed at the previous iteration) to restore flow conservation at the endpoints of e .

Similar to BP for LP, the message functions in BP for $\mathcal{MCF}$, $m_{e \rightarrow v}^t$ for suitable pairs of e and v , are also piecewise-linear convex functions. In §4.1, we establish this fact and present an explicit procedure for computing $m_{e \rightarrow v}^t$. Hence, Algorithm 2 is indeed a procedure that can be implemented on a computer. Next, we state conditions under which the estimates of BP converge to the optimal solution of $\mathcal{MCF}$. Before formally stating the result, we first give the definition of a residual network (Ahuja et al. 1993). Define $G(x)$ to be the residual network of G with respect to flow x as follows: $G(x)$ has the same vertex set as G , $\forall e = (v, w) \in E$ if $x_e < u_e$, then e is an arc in $G(x)$ with cost $c_e^x = c_e$. Finally, if $x_e > 0$, then there is an arc $e' = (w, v)$ in $G(x)$ with cost $c_{e'}^x = -c_e$. Let

$$\delta(x) = \min_{C \in \mathcal{C}} \left\{ c^x(C) = \sum_{e \in C} c_e^x \right\}, \quad (10)$$

where $\mathcal{C}$ is the set of directed cycles in $G(x)$. Note that if x^* is the unique optimal solution of $\mathcal{MCF}$ with directed graph G , then it must be that $\delta(x^*) > 0$ in $G(x^*)$, or else we can change flow x^* along the minimal cost cycle in (10) without increasing its cost.

THEOREM 4.1. Suppose $\mathcal{MCF}$ has a unique optimal solution x^* . Define L to be the maximum cost of a simple directed path in $G(x^*)$. Then, for any $N \geq (\lfloor L/(2\delta(x^*)) \rfloor + 1)n$, $\hat{x}^N = x^*$.

The proof of Theorem 4.1 is presented in §5. The above stated theorem claims that the BP algorithm finds the unique optimal solution of $\mathcal{MCF}$ in at most $(\lfloor L/(2\delta(x^*)) \rfloor + 1)n$ iterations: this convergence is exact in the sense that BP finds the optimal solution exactly in a finite number of iterations. This is in contrast with the asymptotic convergence established for many iterative algorithms in the theory of continuous optimization. We note that this result is similar in flavor to those established in the context of BP’s convergence for combinatorial optimization (Bayati et al. 2008a, b; Sanghavi et al. 2009). However, it differs from the convergence results in Moallemi and Van Roy (2007, 2009) where the estimates converge to the optimal solution with an exponential rate, but are not established to reach an exact optimal in finitely many steps. Next we state the total computation performed by Algorithm 2 to find the optimal solution when the parameters (capacities and costs) are integral in the $\mathcal{MCF}$.

THEOREM 4.2. Given an $\mathcal{MCF}$ with a unique optimal solution x^* and integral data, the BP algorithm finds the unique optimal solution of $\mathcal{MCF}$ in $O(c_{\max}^3 mn^4 \log n)$ operations, where $c_{\max} = \max_e c_e$.

Theorem 4.2 follows by utilizing Theorem 4.1 to bound the number of iterations along with a bound on the number of operations required for updating message functions $m_{e \rightarrow v}^t$ up to those many iterations. The formal proof of this statement is presented in §7.

4.1. Computing/Encoding Message Functions

Here we provide a procedure for constructing message function $m_{e \rightarrow v}^t$ in BP for $\mathcal{MCF}$. This construction procedure shows that each message function $m_{e \rightarrow v}^t$ is a piecewise-linear convex function. Moreover, we provide a bound for the number of operations required for this construction procedure, which will help in bounding the running time of Algorithm 2. First, we formally define piecewise-linear convex function:

DEFINITION 4.3. A function f is called *piecewise-linear convex* if for some finite set of reals, $a_0 < a_1 < \dots < a_n$, (allowing $a_0 = -\infty$ and $a_n = \infty$),

$$f(z) = \begin{cases} c_1(z - a_1) + f(a_1) & \text{if } z \in [a_0, a_1] \\ c_{i+1}(z - a_i) + f(a_i) & \text{if } z \in (a_i, a_{i+1}], 1 \leq i \leq n \\ \infty & \text{otherwise} \end{cases}$$

where $f(a_i) \in \mathbb{R}$ and $c_1 < c_2 < \dots < c_n$ satisfy $c_{i+1}(a_{i+1} - a_i) + f(a_i) = f(a_{i+1})$ for $1 \leq i \leq n-1$.

We define $a_0, a_1, \dots, a_n$ as the vertices of f . We define n to be the number of pieces of f , denoted by $p(f)$. We call $c_i(z - a_{i-1}) + f(a_{i-1})$ for $z \in [a_{i-1}, a_i]$ as the i th linear piece of f . Clearly, if f is a piecewise-linear convex function, then all relevant information about f can be stored using a finite vector of size $O(p(f))$. We make the following observation, which will be useful for efficient update of messages of BP.

OBSERVATION 4.4. Suppose f_1, f_2 are piecewise-linear convex functions. Then, $f_1(ax + b)$, $cf_1(x) + df_2(x)$ are also convex piecewise-linear functions, for any real numbers a, b, c , and d , where $c \geq 0, d \geq 0$.

DEFINITION 4.5. Let $S = \{f_1, f_2, \dots, f_k\}$ be a set of piecewise-linear convex functions, and let $\Psi_t: \mathbb{R}^k \rightarrow \mathbb{R}$ be

$$\Psi_t(x) = \begin{cases} 0 & \text{if } \sum_{i=1}^k x_i = t \\ \infty & \text{otherwise.} \end{cases}$$

Then the interpolation of $f_1, \dots, f_k$ or S , denoted by $I_S(\cdot)$, is defined as

$$I_S(t) = \min_{x \in \mathbb{R}^k} \left\{ \Psi_t(x) + \sum_{i=1}^k f_i(x_i) \right\}, \quad \forall t \in \mathbb{R}.$$

LEMMA 4.6. Suppose f_1, f_2 are piecewise-linear convex functions. Then, for $S = \{f_1, f_2\}$, the $I_S(t)$ is a piecewise-linear convex function, and it can be computed in $O(p(f_1) + p(f_2))$ operations.

PROOF. We shall provide a constructive proof of this result by describing a procedure to construct $I_S(t)$. The idea behind construction of $I_S(t)$ is essentially to “stitch” together the linear pieces of f_1 and f_2 . To this end, let z_1^*, z_2^* be vertices of f_1, f_2 such that $z_1^* = \arg \min f_1(z)$, $z_2^* = \arg \min f_2(z)$. Let $S = \{f_1, f_2\}$. In the case of ties, we select z_i^* to be the smallest point in the arg min set. Let $g(t)$ be

the function that is defined only at $z_1^* + z_2^*$ with $g(z_1^* + z_2^*) = f_1(z_1^*) + f_2(z_2^*)$. Let $L_1 = U_1 = z_1^*$ and $L_2 = U_2 = z_2^*$. We shall construct g iteratively for all $t \in \mathbb{R}$ so that we shall end up with $g(t) = I_S(t)$. The construction is described as follows. At every iteration, let X_1 (and X_2) be the linear piece of f_1 (and f_2) at the left side of L_1 (and L_2). Choose the linear piece with the larger slope from $\{X_1, X_2\}$, and “stitch” this piece onto the left side of the left endpoints of g . If piece, say, P_i , of function f_i is chosen, then update L_i to the vertex that is on the left end of P_i for $i = 1, 2$. As an example, consider f_1 and f_2 shown in the Figure 2. Here, $z_1^* = 1$ and $z_2^* = 0$ are vertices of f_1 and f_2 such that $z_1^* = \arg \min f_1(z)$, $z_2^* = \arg \min f_2(z)$. Note that the linear piece X_1 in the procedure is labeled as $P1$ on the graph, whereas X_2 does not exist (because there is no linear piece for f_2 on the left side of z_2^*). Hence, we “stitch” $P1$ to the left side of g , and update L_1 to 0. In a similar manner, let Y_1 (Y_2) be the linear piece of f_1 (f_2) to the right side of U_1 (U_2). Then choose the linear piece with the smaller slope and “stitch” this piece onto the right side of the right endpoint of g . If Q_i is the chosen piece, update U_i to the vertex that is on the right side of Q_i for $i = 1, 2$. Again, we use f_1 and f_2 in Figure 2 as an illustration. The linear piece Y_1 in the procedure is labeled as $P2$, whereas Y_2 is labeled as $P3$. Because $P2$ has a lower slope than $P3$, we “stitch” $P2$ to the right side of g and update U_1 to 2.

Repeat this procedure until both L_1 (and L_2) and U_1 (and U_2) are the leftmost (and rightmost) endpoints of f_1 (and f_2), or both endpoints of g are infinity. See Figures 2 and 3 as an illustration of resulting interpolation of the two functions.

Note that the total number of iterations is bounded by $O(p(f_1) + p(f_2))$ and each iteration takes at most constant number of operations. Thus, total computation performed to obtain g is $O(p(f_1) + p(f_2))$. By construction, it is clear that g is a piecewise-linear convex function. Also, $g(z_1^* + z_2^*) = f_1(z_1^*) + f_2(z_2^*)$, and by the way we have constructed g , we must have $g(t) \leq \{\Psi_t(x) + f_1(x_1) +$

Figure 2. Functions f_1 and f_2 .

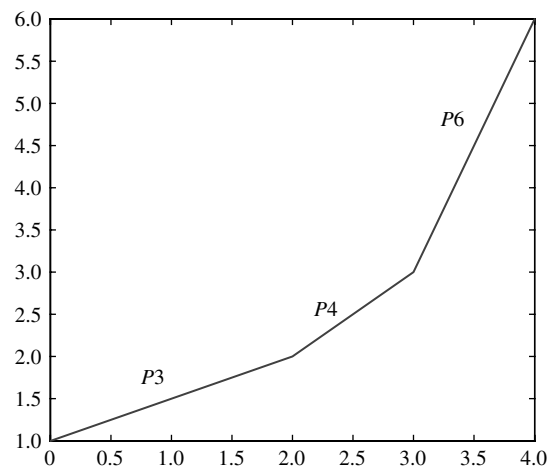
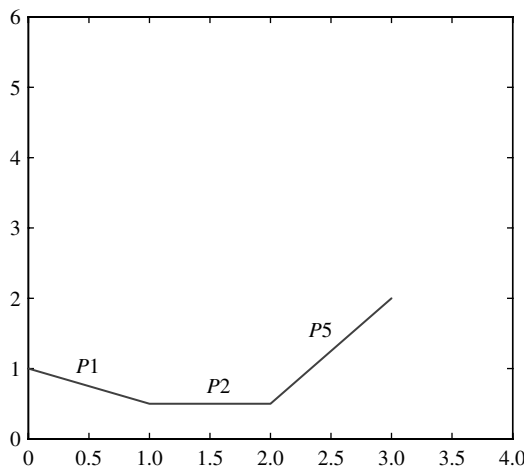
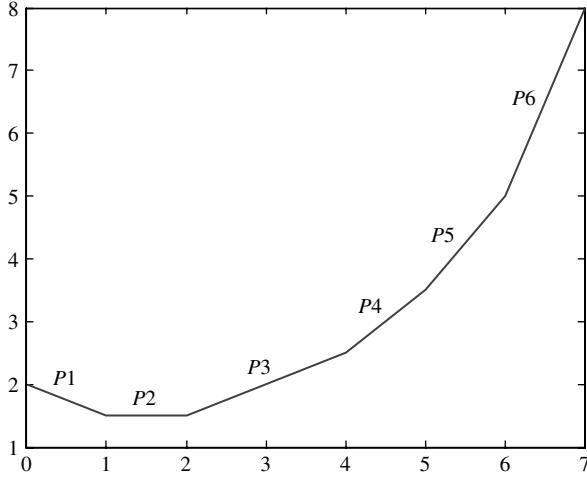


Figure 3. Interpolation of f_1 and f_2 .

$f_2(x_2)\}$ for any $t \in \mathbb{R}$. Therefore, it follows that $g = I_S$. This completes the proof of Lemma 4.6. $\square$

THEOREM 4.7. Given a set $S\{f_1, \dots, f_k\}$ of piecewise-linear convex functions, $I_S(t)$ is also a piecewise-linear convex function. Let $P = \sum_{f \in S} p(f)$. Then $I_S(t)$ can be computed in $O(P \log k)$ operations.

PROOF. Without the loss of generality, we may assume that k is divisible by 2. Let $S_1 = \{f_1, f_2\}$, $S_2 = \{f_3, f_4\}$, $\dots$, $S_{k/2} = \{f_{k-1}, f_k\}$, and $S' = \{I_{S_1}, I_{S_2}, \dots, I_{S_{k/2}}\}$. Then one can observe that $I_{S'} = I_S$ by the definition of I_S . By Lemma 4.6, each function in S' is piecewise-linear convex and S' can be computed in $O(P)$ operations. Consider changing S to S' as a procedure of decreasing the number of piecewise-linear convex functions. This procedure reduces the number by a factor of 2 each time while it consumes $O(P)$ operations. Hence, it takes $O(\log k)$ procedures to reduce set S into a single piecewise-linear convex function; and hence, computing $I_S(t)$ takes $O(P \log k)$ operations. $\square$

DEFINITION 4.8. Let $S = \{f_1, f_2, \dots, f_k\}$ be a set of convex piecewise-linear functions, $a \in \mathbb{R}^k$, and let $\Psi_t: \mathbb{R}^k \rightarrow \mathbb{R}$ be

$$\Psi_t(x) = \begin{cases} 0 & \text{if } \sum_{i=1}^k a_i x_i = t \\ \infty & \text{otherwise.} \end{cases} \quad \forall v \in V,$$

We call $I_S^a(t) = \min_{x \in \mathbb{R}^k} \{\Psi_t(x) + \sum_{i=1}^k f_i(x_i)\}$ the scaled interpolation of S .

THEOREM 4.9. Given a set of piecewise-linear convex functions $S = \{f_1, \dots, f_k\}$, $I_S^a(t)$ is also a piecewise-linear convex function. Let $P = \sum_{f \in S} p(f)$. Then $I_S(t)$ can be computed in $O(P \log k)$ operations.

PROOF. Let $S = \{f_1, \dots, f_k\}$ and $S' = \{f'_1, \dots, f'_k\}$ with $f'_i(x) = f_i(a_i x)$ for $1 \leq i \leq k$. If f_i is a piecewise linear

convex function, then it can be easily checked that so is f'_i for $1 \leq i \leq k$. Therefore, Theorem 4.9 follows immediately by an application of Theorem 4.7 to S' . $\square$

Now recall that for any $t \geq 1$, the message update in the BP for $\mathcal{MCF}$ problem has the following form:

$$m_{e \rightarrow v}^t(z) = \phi_e(z) + \min_{\tilde{z} \in \mathbb{R}^{|E_w|}, \tilde{z}_e = z} \left\{ \psi_w(\tilde{z}) + \sum_{\tilde{e} \in E_w \setminus e} m_{\tilde{e} \rightarrow w}^{t-1}(\tilde{z}_{\tilde{e}}) \right\} \quad \text{for } z \in \mathbb{R}.$$

Therefore, the message update can be performed using the scaled interpolation. Specifically, we make the following observation.

OBSERVATION 4.10. Let $S = \{m_{\tilde{e} \rightarrow w}^{t-1}, \tilde{e} \in E_w \setminus e\}$ and $a_{\tilde{e}} = \Delta(w, \tilde{e})$ for any $\tilde{e} \in E_w \setminus e$. Then the function $\tilde{m}_{e \rightarrow v}^t(z) = m_{e \rightarrow v}^t(z) - \phi_e(z)$ is equal to $I_S^a(-\Delta(w, e)z + f_w)$.

From Observation 4.10 above, the following corollaries are immediate.

COROLLARY 4.11. Any message function $m_{e \rightarrow v}^t$ of the BP algorithm for $\mathcal{MCF}$ is piecewise linear convex for any $t \geq 0$ and any arc e , vertex v , where e is incident to v .

PROOF. The proof follows by induction on t . For $t = 0$, $m_{e \rightarrow v}^0$ is a constant (and hence piecewise linear) function equal to 0 for any e and v where e is incident to v . For $t \geq 1$, by induction hypothesis, assume message functions in the form of $m_{\tilde{e} \rightarrow w}^{t-1}$ are piecewise linear. By Theorem 4.9 and Observation 4.10, for any e and v where e is incident to v , $m_{e \rightarrow v}^t(z) - \phi_e(z)$ is piecewise linear convex. As ϕ_e is also piecewise linear convex, $m_{e \rightarrow v}^t$ is a summation of two piecewise linear convex functions which is piecewise linear convex as well. $\square$

COROLLARY 4.12. Suppose the components of cost vector c in $\mathcal{MCF}$ are integers. At iteration t , for piecewise-linear convex message function $m_{e \rightarrow v}^t(z)$ of the BP algorithm for $\mathcal{MCF}$, let $\{s_1, s_2, \dots, s_k\}$ be the slopes of its pieces. Then $-tc_{\max} \leq s_i \leq tc_{\max}$ and s_i is integral for each $1 \leq i \leq k$, where $c_{\max} = \max_e c_e$.

PROOF. The proof follows by induction on t . Initially, $t = 0$ and the statement is immediate. For $t \geq 1$, because $\Delta(w, e) = \pm 1$ for any $e \in E_w$, by Observation 4.10 it follows that the absolute values of the slopes for the linear pieces of $m_{e \rightarrow v}^t - \phi_e$ is the same as the absolute values of the slopes for the linear pieces of message functions $m_{\tilde{e} \rightarrow w}^{t-1}$. By the induction hypothesis, the absolute values of the slopes of $m_{\tilde{e} \rightarrow w}^{t-1} - \phi_{\tilde{e}}$ are integral and bounded by $(t-1)c_{\max}$. The slope of pieces in ϕ_e is c_e , and therefore, the absolute values of slopes of $m_{e \rightarrow v}^t$ are integral and bounded by $tc_{\max}$. $\square$

COROLLARY 4.13. Suppose components of vectors f and u take integer values in $\mathcal{MCF}$. Then at iteration $t \geq 1$, for any message function $m_{e \rightarrow v}^t$, the vertices of $m_{e \rightarrow v}^t$ are integral as well.

PROOF. Again, the proof is by induction on t . Initially, $t=0$, and the statement trivially holds. For $t \geq 1$, first observe that because u has integral components, all of its vertices of ϕ_e are integral as well. By Observation 4.10 and induction hypothesis, all vertices of $m'_{e \rightarrow v} - \phi_e$ are integral. Therefore, all vertices of $m'_{e \rightarrow v}$ are integral. $\square$

Theorem 4.9 and Corollary 4.11 show that at every iteration, each message function can be encoded in terms of a finite vector describing the corners and slopes of its linear pieces in a finite number of iterations. These arguments extend easily to the form of linear program considered earlier; that is, BP for LP can be truly implemented on a computer.

Corollary 4.12 provides a bound for the number of linear pieces in $m'_{e \rightarrow v}$. This bound will help us bound the running time of BP algorithm for $\mathcal{MCF}$. We shall discuss this in detail in §7. Finally, we would like to note that the result that message functions $m'_{e \rightarrow v}$ are piecewise-linear convex functions can be also shown by sensitivity analysis of LP, cf. Bertsimas and Tsitsiklis (1997, Chapter 5).

4.2. BP for a Subclass of $\mathcal{MCF}$

Section 4.1 established that each message function is a piecewise-linear convex function. However, as per the bounds established, the number of pieces increases linearly with iterations, and this requires more computation for message update as iterations grow. Now, for an instance of $\mathcal{MCF}$ with integral components of vector b and u , the message function $m'_{e \rightarrow v}$ is a piecewise-linear convex function with integral vertices as per Corollary 4.13. Therefore, it has at most u_e linear pieces. Thus, if u_e is bounded by some constant for all e , the message functions at every iteration are piecewise-linear convex functions with a bounded number of pieces. This results in a computationally efficient update of messages. Next we present a subclass of $\mathcal{MCF}$, denoted by $\mathcal{MCF}^o$, for which such a property holds and which contains important classes of network flow problems.

To this end, given a directed graph $G = (V, E)$, consider the following subclass of problem: with notation $\text{in}(v) = \{(u, v) \in E\}$

$$\begin{aligned} & \text{minimize} \quad \sum_{e \in E} c_e x_e \\ & \text{subject to} \quad \sum_{e \in \text{in}(v)} \Delta(v, e) x_e = f_v, \quad \forall v \in V \\ & \quad \quad \quad (\text{demand/supply constraints}) \\ & \quad \quad \quad \sum_{e \in \text{in}(v)} x_e \leq \tilde{u}_v, \quad \forall v \in V \\ & \quad \quad \quad 0 \leq x_e \leq u_e, \quad \forall e \in E \quad (\text{flow constraints}). \end{aligned}$$

Above, c , u , and $\tilde{u}$ are all integral. To see that $\mathcal{MCF}^o$ is indeed an instance of $\mathcal{MCF}$, consider the following. Split each $v \in V$ into two vertices v_{in} and v_{out} , where v_{in} is incident to all in-arcs of v with $f_{v_{in}} = 0$ and v_{out} is incident to all out-arcs of v with $f_{v_{out}} = f_v$. Create an arc from v_{in} to v_{out}

with capacity $\tilde{u}_v$ and cost equal to 0. Denote this newly created graph by G^o . Then the $\mathcal{MCF}$ on G^o is equivalent to $\mathcal{MCF}^o$. Instead of using Algorithm 2 to solve the $\mathcal{MCF}$ on G^o , we shall use it on G with the following functions ψ , ϕ :

$$\psi_v(x) = \begin{cases} 0 & \text{if } \sum_{e \in E_v} \Delta(v, e) x_e = f_v \text{ and} \\ & \sum_{e \in \text{in}(v)} x_e \leq \tilde{u}_v \\ \infty & \text{otherwise,} \end{cases} \quad \forall v \in V,$$

$$\phi_e(x) = \begin{cases} c_e x & \text{if } 0 \leq x \leq u_e \\ \infty & \text{otherwise,} \end{cases} \quad \forall e \in E.$$

Now, to update message functions $m'_{e \rightarrow v}$ for all $e \in E_w$, the inequality $\sum_{e \in \text{in}(w)} x_e \leq \tilde{u}_w$ implies that it is sufficient to check $\tilde{u}_w$ linear pieces from message functions $m'_{e \rightarrow w}$ for all but a constant number of $e \in E_w$. This leads to efficient implementation of BP for $\mathcal{MCF}^o$. Specifically, we state the following result.

THEOREM 4.14. *Suppose the $\mathcal{MCF}^o$ as described above has a unique optimal solution with*

$$\max_v (\tilde{u}_v, u_v, |f_v|) \leq K, \quad \max_e c_e \leq K.$$

Then Algorithm 2 for $\mathcal{MCF}^o$ finds the unique optimal solution using $O(K^2 m n^2 \log n)$, which is $O(K^2 n^4 \log n)$ operations in total. As a result, Algorithm 2 is polynomial time when K is a constant.

The proof of Theorem 4.14 is presented in §7.1. It is worth taking note of the fact that both the shortest-path problem and maximum weight matching in a bipartite graph belong to the $\mathcal{MCF}^o$ class of problems with all components of f , u being bounded by 2. For these two classes of problems we do not need the extra constraint $\sum_{e \in \text{in}(v)} x_e \leq \tilde{u}_v$, but we do need this constraint to make a general statement of the theorem. We see that under the uniqueness assumptions, BP solves these problems in polynomial (as opposed to just pseudopolynomial) time.

5. Convergence of BP for $\mathcal{MCF}$

This section is devoted to establishing the convergence of BP to the optimal solution of the $\mathcal{MCF}$ under the assumption of the uniqueness of the optimal solution; namely, we shall prove Theorem 4.1. The outline of the proof is as follows. First, we define the notion of a computation tree T_e^N that is associated with each variable node x_e of $\mathcal{MCF}$ for iteration N . We show that in fact the estimation $\hat{x}_e^N$ under BP is the optimal solution of an appropriately defined $\mathcal{MCF}$ problem on T_e^N (Lemma 5.1). Next, we show that the optimal assignment to x_e under the min-cost flow problem on the computation tree T_e^N is the same as the optimal assignment to x_e under the original $\mathcal{MCF}$ as long as N

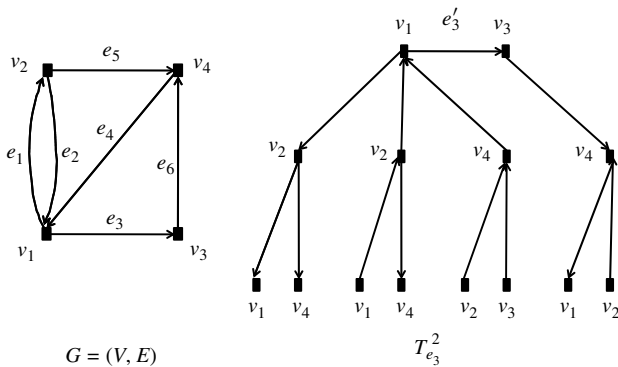
is large enough (see §5.2). This immediately implies that BP finds the correct optimal solution for $\mathcal{MCF}$ for large enough N leading to Theorem 4.1. We note that this strategy is similar to that of Bayati et al. (2008a). However, the technical details are quite different.

5.1. Computation Tree and BP

We start with the definition of a computation tree. The N -level computation tree associated with arc $e = (v, w) \in E$ is denoted by T_e^N . It is essentially the breadth-first search tree of G (with repetition of nodes allowed) starting from e up to depth N . Formally, computation tree T_e^N is defined inductively as follows. $T_e^0 = (V(T_e^0), E(T_e^0))$ is a tree with vertex set $V(T_e^0) = \{v', w'\}$ and arc set $E(T_e^0) = \{e' = (v', w')\}$. The v', w' are considered replicas of $v, w \in V$, and this is represented by a mapping $\Gamma_e^0: V(T_e^0) \rightarrow V$ with $\Gamma_e^0(v') = v$ and $\Gamma_e^0(w') = w$. The arc e' is considered the “root” of T_e^0 , and vertices v', w' are considered to be at level 0. Inductively, let us suppose that tree $T_e^N = (V(T_e^N), E(T_e^N))$ is defined with corresponding $\Gamma_e^N: V(T_e^N) \rightarrow V$ such that for $u'_1, u'_2 \in V(T_e^N)$, $(u'_1, u'_2) \in E(T_e^N)$ only if $(\Gamma_e^N(u'_1), \Gamma_e^N(u'_2)) \in E$. Let $P: V(T_e^N) \rightarrow V(T_e^N)$ represent the parent relation in T_e^N . And for the sake of simplicity, let w' be the parent of v' ($P(v') = w'$) and v' be the parent of w' ($P(w') = v'$). Let $L(T_e^N)$ be the set of leaves¹ of T_e^N . Now we shall define $T_e^{N+1} = (V(T_e^{N+1}), E(T_e^{N+1}))$, which contains T_e^N as a subtree. Specifically, $V(T_e^{N+1})$ and $E(T_e^{N+1})$ are obtained by adding vertices to $V(T_e^N)$ and arcs to $E(T_e^N)$ as follows. For each leaf node $u' \in L(T_e^N)$, add node $\tilde{u}'$ to expand $V(T_e^N)$ and add arc $(u', \tilde{u}')$ or $(\tilde{u}', u')$ to expand $E(T_e^N)$ if (a) there is a node $\tilde{u} \in V$ so that $(u, \tilde{u})$ or $(\tilde{u}, u)$ is in E with $\Gamma_e^N(u') = u$, and (b) $\Gamma_e^N(P(u')) \neq \tilde{u}$. In this case, define $P(\tilde{u}') = u'$, the map $\Gamma_e^{N+1}(\tilde{u}') = \tilde{u}$, and level of $\tilde{u}'$ as $N + 1$. Indeed, Γ_e^{N+1} is identical to Γ_e^N for nodes $V(T_e^N) \subset V(T_e^{N+1})$. In what follows, we shall drop reference to e, N in notation of Γ_e^N when clear from context and abuse notation by denoting $\Gamma(e' = (u'_1, u'_2)) = (\Gamma(u'_1), \Gamma(u'_2))$.

Sometimes T_e^N is also called “unwrapped tree” of G rooted at e . Figure 4 gives an example of a computation

Figure 4. Computation tree of G rooted at $e_3 = (1, 3)$.



tree. It should be noted that the definition of a computation tree may appear slightly different compared to that in related works such as Bayati et al. (2008a, b) and Sanghavi et al. (2007) (arc is the root here, in contrast to a vertex as the root). However, the utility of the computation trees is very similar.

Now we are ready to relate the computation tree with the BP. Let $V^o(T_e^N) \subset V(T_e^N)$ denote the set of all the vertices that are not on the N th level of T_e^N . Consider the following problem:

$$\begin{aligned} & \text{minimize} && \sum_{\tilde{e} \in E(T_e^N)} c_{\Gamma(\tilde{e})} x_{\tilde{e}} \\ & \text{subject to} && \sum_{\tilde{e} \in E_{u'}} \Delta(u', \tilde{e}) x_{\tilde{e}} = f_{\Gamma(u')}, \quad \forall u' \in V^o(T_e^N) \\ & && 0 \leq x_{\tilde{e}} \leq u_{\Gamma(f)}, \quad \forall f \in E(T_e^N). \end{aligned} \quad (\mathcal{MCF}_e^N)$$

Above, $E_{u'} \subset E(T_e^N)$ is the set of arcs incident on $u' \in V^o(T_e^N)$ in T_e^N and $\Delta(u', \tilde{e})$ for $\tilde{e} \in E_{u'}$ is defined as -1 or $+1$, depending upon whether e' is in-arc or out-arc for node u' . Loosely speaking, $\mathcal{MCF}_e^N$ is essentially an $\mathcal{MCF}$ on T_e^N : there is a flow constraint for every arc $\tilde{e} \in E(T_e^N)$ and a demand/supply constraint for every node, except for the nodes on the N th level. Now we state the following well-known result that exhibits the connection between BP and the computation trees.

LEMMA 5.1. Let $\hat{x}_e^N$ be the value produced by BP at the end of iteration N for the flow value on edge $e \in E$. Then there exists an optimal solution y^* of $\mathcal{MCF}_e^N$ such that $y_{e'}^* = \hat{x}_e^N$ where e' is the root of T_e^N (and $\Gamma(e') = e$).

PROOF. Let $e' = (v', w')$ be the root arc of computation tree T_e^N with $e = (v, w)$, such that $\Gamma(e') = e$, $\Gamma(v') = v$, and $\Gamma(w') = w$. By definition, T_e^N has two components connected via the root arc e' . Let C be the component containing w' and $T_{e' \rightarrow v'}^N$ denote $C \cup e'$; indeed, $T_{e' \rightarrow v'}^N$ is a tree. As before, let $V^o(T_{e' \rightarrow v'}^N)$ be the set of all nodes, excluding those at the N th level. Define

$$\begin{aligned} & \text{minimize} && \sum_{\tilde{e} \in E(T_{e' \rightarrow v'}^N)} c_{\Gamma(\tilde{e})} x_{\tilde{e}} \\ & \text{subject to} && \sum_{\tilde{e} \in E_{q'}} \Delta(q', \tilde{e}) x_{\tilde{e}} = f_{\Gamma(q')}, \quad \forall q' \in V^o(T_{e' \rightarrow v'}^N) \\ & && x_{e'} = z, \\ & && 0 \leq x_{\tilde{e}} \leq u_{\Gamma(\tilde{e})}, \quad \forall \tilde{e} \in E(T_{e' \rightarrow v'}^N). \end{aligned} \quad (\mathcal{MCF}_{e' \rightarrow v'}^N(z)).$$

Now we shall establish that under the BP algorithm (running on G) the value of message function from $e \rightarrow v$ evaluated at z , that is, $m_{e \rightarrow v}^N(z)$, is the same as the cost of the optimal assignment for $\mathcal{MCF}_{e' \rightarrow v'}^N(z)$. This can be established inductively. To start with, for $N = 1$, the statement can be checked to be true trivially. For $N > 1$, let $E_{w'}$ denote the edges incident on w' in T_e^N , where recall

that $e' = (v', w)$ is its root arc. Then for each $g' \in E_{w'} \setminus e'$ with $g' = (u', w')$ (or (w', u')), let $T_{g' \rightarrow w'}^{N-1}$ be the subtree of $T_{e' \rightarrow v'}^N$ that includes g' and everything in $T_{e' \rightarrow v'}^N$ that is part of its component that does not include w' . Define the optimization problem:

$$\begin{aligned} & \text{minimize} && \sum_{\tilde{e} \in E(T_{g' \rightarrow w'}^{N-1})} c_{\Gamma(\tilde{e})} x_{\tilde{e}} \\ & \text{subject to} && \sum_{\tilde{e} \in E_{q'}} \Delta(q', \tilde{e}) x_{\tilde{e}} = f_{\Gamma(q')} \quad \forall q' \in V^o(T_{g' \rightarrow w'}^{N-1}) \\ & && x_{g'} = z, \\ & && 0 \leq x_{\tilde{e}} \leq u_{\Gamma(\tilde{e})}, \quad \forall \tilde{e} \in E(T_{g' \rightarrow w'}^{N-1}). \end{aligned}$$

By induction hypothesis, it must be that $m_{g' \rightarrow w'}^{N-1}(z)$ equals the cost of the solution of $\mathcal{MEF}_{g' \rightarrow w'}^{N-1}(z)$. Given this hypothesis and the relation of subtree $T_{g' \rightarrow w'}^{N-1}$ for all $g' \in E_{w'} \setminus e'$ with $T_{e' \rightarrow v'}^N$, it follows that the optimization problem $\mathcal{MEF}_{e' \rightarrow v'}^N(z)$ is equivalent to

$$\begin{aligned} & \text{minimize} && c_e z + \sum_{g' \in E_{w'} \setminus e'} m_{\Gamma(g') \rightarrow \Gamma(w')}^{N-1}(x_{g'}) \\ & \text{subject to} && \Delta(w', e') z + \sum_{g' \in E_{w'} \setminus e'} \Delta(w', g') x_{g'} = f_{\Gamma(w')} \\ & && 0 \leq x_{g'} \leq u_{\Gamma(g')}, \quad \forall g' \in E_{w'} \setminus e'. \end{aligned}$$

This is exactly the same as the relation between $m_{e \rightarrow v}^N(z)$ and message function $m_{g \rightarrow w}^{N-1}(\cdot)$ for $g \in E_w \setminus e$ as defined by BP; that is, $m_{e \rightarrow v}^N(z)$ is exactly the same as the cost of optimal assignment of $\mathcal{MCF}_{e' \rightarrow v'}^N$. We shall use this equivalence to complete the proof of Lemma 5.1.

To that end, for given $e = (v, w)$ with $0 \leq z \leq u_e$, the optimization problem $\mathcal{MCF}_e^N(z)$ is equivalent to

$$\begin{aligned} & \text{minimize} \quad c_e z + \sum_{\tilde{e} \in E(T_{e'}^N \rightarrow v')} c_{\Gamma(\tilde{e})} x_{\tilde{e}} + \sum_{\tilde{e} \in E(T_{e'}^N \rightarrow w')} c_{\Gamma(\tilde{e})} x_{\tilde{e}} \\ & \text{subject to} \quad \sum_{\tilde{e} \in E_{q'}} \Delta(q', \tilde{e}) x_{\tilde{e}} = f_{\Gamma(q')}, \\ & \quad \quad \quad \forall q' \in V^o(T_e^N) \cap (V(T_{e'}^N \rightarrow v') \cup V(T_{e'}^N \rightarrow w')) \\ & \quad \quad \quad 0 \leq x_{\tilde{e}} \leq u_{\Gamma(\tilde{e})}, \quad \tilde{e} \in E(T_{e'}^N \rightarrow v') \cup E(T_{e'}^N \rightarrow w'); \end{aligned}$$

that is, the cost of an optimal assignment of $\mathcal{MCF}_e^N(z)$ equals $m_{e \rightarrow u}^N(z) + m_{e \rightarrow v}^N(z) + c_e z$ for any $0 \leq z \leq u_e$. Now the claim of Lemma 5.1 follows immediately. $\square$

5.2. Proof of Theorem 4.1

Now we are ready to establish Theorem 4.1. Suppose to the contrary that there exists $e_0 = (v_\alpha, v_\beta) \in E$ and $N \geq \lceil L/(2\delta(x^*)) \rceil + 1$ such that $\hat{x}_{e_0}^N \neq x_{e_0}^*$. By Lemma 5.1, there exists an optimal solution y^* of $\mathcal{ME}_{e_0}^N$ such that $y_{e_0}^* = \hat{x}_{e_0}^N$. Without loss of generality, assume $y_{e_0}^* > x_{e_0}^*$. Using the optimality of x^* , we will show that it is possible to modify y^* to obtain a feasible solution of $\mathcal{ME}_{e_0}^N$.

with cost strictly lower than that of y^* . This will lead to contradiction to the assumption that $\hat{x}_{e_0}^N \neq x_{e_0}^*$, and establish the result.

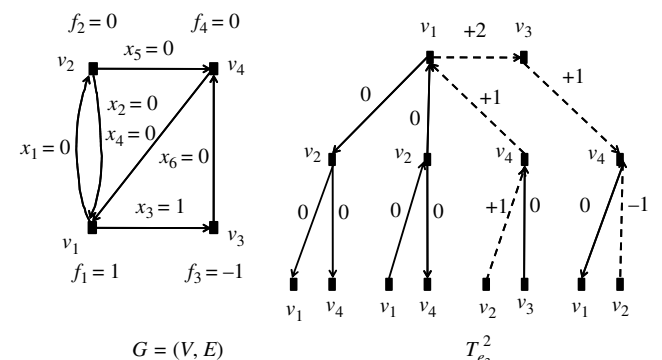
To that end, let $e'_0 = (v'_\alpha, v'_\beta)$ be the root edge of the computation tree $T_{e'_0}^N$ as discussed earlier. Because y^* is a feasible solution of $\mathcal{MEF}_{e'_0}^N$ and x^* is a feasible solution of $\mathcal{MEF}$,

$$\begin{aligned}
f_{\Gamma(v'_\alpha)} &= \sum_{\tilde{e} \in E_{v'_\alpha}} \Delta(v'_\alpha, \tilde{e}) y_{\tilde{e}}^* = y_{e'_0}^* + \sum_{\tilde{e} \in E_{v'_\alpha} \setminus e'_0} \Delta(v'_\alpha, \tilde{e}) y_{\tilde{e}}^* \\
&\quad \text{(constraint at } v'_\alpha \text{ in } \mathcal{MEF}_{e'_0}^N) \\
f_{\Gamma(v'_\alpha)} &= \sum_{\tilde{e} \in E_{\Gamma(v'_\alpha)}} \Delta(\Gamma(v'_\alpha), \tilde{e}) x_{\tilde{e}}^* = x_{e_0}^* + \sum_{\tilde{e} \in E_{\Gamma(v'_\alpha)} \setminus e_0} \Delta(\Gamma(v'_\alpha), \tilde{e}) x_{\tilde{e}}^* \\
&\quad \text{(constraint at } \Gamma(v'_\alpha) \text{ in } \mathcal{MEF}).
\end{aligned}$$

Note that the edges in $E_{v'_\alpha}$ in the computation tree $T_{e'_0}^N$ are copies of edges in E_{v_α} in G where $v_\alpha = \Gamma(v'_\alpha)$. Therefore, $\Delta(v'_\alpha, \tilde{e}) = \Delta(\Gamma(v_\alpha), \Gamma(\tilde{e}))$ for $\tilde{e} \in E_{v'_\alpha}$. Therefore, from the above inequalities, it follows that because $y_{e'_0}^* > x_{e'_0}^*$, there exists arc $e'_1 \neq e'_0$ incident on v'_α in $T_{e'_0}^N$ such that $\Delta(v'_\alpha, e'_1)(x_{\Gamma(e'_1)}^* - y_{e'_1}^*)$ is strictly positive. Therefore, if $\Delta(v'_\alpha, e'_1) = 1$, then $x_{\Gamma(e'_1)}^* > y_{e'_1}^*$, or else $x_{\Gamma(e'_1)}^* < y_{e'_1}^*$. That is, if edge e'_1 has the opposite orientation with respect to e'_0 at node v'_α (both are outgoing from v'_α , and hence, opposite orientation), then $x_{\Gamma(e'_1)}^* > y_{e'_1}^*$ or else $x_{\Gamma(e'_1)}^* < y_{e'_1}^*$. Figure 5 explains this by means of a simple example.

More generally, using a similar argument, we can find arc $e'_{-1} \neq e'_0$ incident to v'_β satisfying a similar condition. Let v'_{α_1} , $v'_{\alpha_{-1}}$ be the other endpoints of e'_1 , e'_{-1} , respectively. A recursive application of a similar argument utilizing the feasibility condition of x^* and y^* and the inequalities between the value of components of x^* and y^* at edges e'_1 and e'_{-1} leads to the existence of arcs e'_2 , e'_{-2} incident on v'_{α_1} , $v'_{\alpha_{-1}}$, respectively, so that $x^*_{e'_2} \neq y^*_{e'_2}$ and $x^*_{e'_{-2}} \neq y^*_{e'_{-2}}$ with inequalities being $<$ or $>$ depending upon the orientation of the edges with respect to e_0 . Continuing further in

Figure 5. An example of augmenting path between the flow assignment on computation tree $T_{e_3}^2$ and the flow assignment on G .



Note. The dashed edges represent the edges belonging to the augmenting path. Root edge and edge from v_4 to v_1 have the same orientation.

this manner all the way down to the leaves, it is possible to find arcs $\{e'_{-N}, e'_{-N+1}, \dots, e'_{-1}, e'_1, \dots, e'_N\}$ such that for $-N \leq i \leq N$,

$$y_{e'_i}^* > x_{\Gamma(e'_i)}^* \iff e'_i \text{ has the same orientation as } e_0,$$

$$y_{e'_i}^* < x_{\Gamma(e'_i)}^* \iff e'_i \text{ has the opposite orientation as } e_0.$$

Let us denote the path containing these edges as $X = \{e'_{-N}, e'_{-N+1}, \dots, e'_{-1}, e'_0, e'_1, \dots, e'_N\}$. For any $e' = (v'_p, v'_q) \in X$, define $\text{Aug}(e') = (v'_p, v'_q)$ if $y_{e'}^* > x_{\Gamma(e')}^*$, and $\text{Aug}(e') = (v'_q, v'_p)$ if $y_{e'}^* < x_{\Gamma(e')}^*$. Given the feasibility conditions of y^* and definition of $\text{Aug}(e')$, it can be checked that $\Gamma(\text{Aug}(e'))$ is an arc in the residual graph $G(x^*)$. The directed path $W = (\text{Aug}(e'_{-N}), \dots, \text{Aug}(e'_0), \dots, \text{Aug}(e'_N))$ on $T_{e_0}^N$ will be called the *augmenting* path of y^* with respect to x^* . Also, $\Gamma(W)$ is a directed walk on $G(x^*)$. Now we can decompose $\Gamma(W)$ into a simple directed path P and a collection of simple directed cycles $C_1, \dots, C_k$. Now each simple directed cycle or path on $G(x^*)$ can have at most n edges. Because W has $2N + 1$ arcs and $N \geq (\lfloor L/(2\delta(x^*)) \rfloor + 1)n$, it follows that $k > L/(\delta(x^*))$. Now the cost of path P , denoted by $c^*(P)$ with respect to the residual graph $G(x^*)$, is at least $-L$ (and at most L) by definition of L . Because each C_i is a simple cycle in $G(x^*)$, by definition its cost, denoted by $c^*(C_i)$ with respect to $G(x^*)$, is at least $\delta(x^*)$; $\delta(x^*) > 0$ because x^* is the unique optimal solution. Therefore, as explained below, we obtain that the cost of W is strictly positive:

$$\begin{aligned} \sum_{i=-N}^N c_{\Gamma(e'_i)}^* &= c^*(W) \\ &= c^*(P) + \sum_{j=1}^k c^*(C_j) \\ &\geq -L + k\delta(x^*) \\ &> -L + \frac{L}{\delta(x^*)}\delta(x^*) = 0. \end{aligned}$$

Let $\text{FWD} = \{e \in X: y_e^* > x_{\Gamma(e)}^*\}$, $\text{BCK} = \{e \in X: y_e^* < x_{\Gamma(e)}^*\}$. Because both FWD and BCK are finite, there exists $\lambda > 0$ such that $y_e^* - \lambda \geq x_{\Gamma(e)}^*$, $\forall e \in \text{FWD}$ and $y_e^* + \lambda \leq x_{\Gamma(e)}^*$, $\forall e \in \text{BCK}$. Define $\tilde{y} \in \mathbb{R}^{|E(T_{e_0}^N)|}$ as

$$\tilde{y}_e = \begin{cases} y_e^* - \lambda & e \in \text{FWD} \\ y_e^* + \lambda & e \in \text{BCK} \\ 0 & \text{otherwise.} \end{cases}$$

The $\tilde{y}$ can be thought of as flow that is obtained by pushing λ units of additional flow along path W over the existing flow y^* in $T_{e_0}^N$. Because for each $e \in \text{FWD}$, $y_e^* - \lambda \geq x_{\Gamma(e)}^* \geq 0$, and for each $e \in \text{BCK}$, $y_e^* + \lambda \leq x_{\Gamma(e)}^* \leq u_{\Gamma(e)}$, $\tilde{y}$ satisfies all the flow constraints. Further, because all edges in FWD have the same orientation as e_0 and those in BCK have the

opposite orientation compared to e_0 , we have that for any $v' \in V^o(T_{e_0}^N)$,

$$\begin{aligned} \sum_{e' \in E_{v'}} \Delta(v', e') \tilde{y}_{e'} &= \sum_{v' \in E_{v'}} \Delta(v', e') y_{e'}^* \\ &= f_{\Gamma(v')}, \end{aligned}$$

which implies that $\tilde{y}$ satisfies all the demand/supply constraints. Therefore, $\tilde{y}$ is a feasible solution of $\mathcal{MCF}_{e_0}^N$. Now

$$\begin{aligned} \sum_{e' \in E(T_{e_0}^N)} c_{\Gamma(e')} y_{e'}^* - \sum_{e' \in E(T_{e_0}^N)} c_{\Gamma(e')} \tilde{y}_{e'} \\ &= \sum_{e' \in E(T_{e_0}^N)} c_{\Gamma(e')} (y_{e'}^* - \tilde{y}_{e'}) \\ &= \sum_{e' \in \text{FWD}} c_{\Gamma(e')} \lambda - \sum_{e' \in \text{BCK}} c_{\Gamma(e')} \lambda \\ &= c^*(W) \lambda \\ &> 0. \end{aligned}$$

Above we have used the fact that $c_{\Gamma(e')}^* = c_{\Gamma(e')}$ for $e' \in \text{FWD}$ and $c_{\Gamma(e')}^* = -c_{\Gamma(e')}$ for $e' \in \text{BCK}$. The above contradicts the optimality of y^* . Therefore, the assumption about the BP estimate not converging is false. This completes the proof of Theorem 4.1.

5.3. Detection of Uniqueness of the Optimal Solution Using BP

In this section, we establish an unusual property of BP in terms of its ability to detect the uniqueness of the optimal solution in the $\mathcal{MCF}$ in a distributed manner as long as the input parameters c , f , and u are integral. We state this as the following corollary of Theorem 4.1.

COROLLARY 5.2. Consider an instance of $\mathcal{MCF}$ with integral c , f , and u . Suppose $c_{\max} = \max_{e \in E} c_e$. Suppose the BP Algorithm 2 runs for $N = n^2 c_{\max} + n$ iterations. Let $z_e^* \in \arg \min b_e^N(z)$. Then

$$\begin{aligned} \forall e \in E, \quad \min(b_e^N(z_e^* - 1), b_e^N(z_e^* + 1)) \\ > nc_{\max} + b_e^N(z_e^*) \end{aligned} \quad (11)$$

if and only if the $\mathcal{MCF}$ instance has a unique solution.

PROOF. We first establish the implication that if $\mathcal{MCF}$ has a unique optimal solution, then (11) holds. To that end, let us suppose that the instance of $\mathcal{MCF}$ of interest has a unique solution. Consider any edge $e \in E$ and its computation tree T_e^N . Then from Lemma 5.1, it follows that z_e^* is an optimal assignment of the root edge e' of T_e^N with respect to the associated optimization problem $\mathcal{MCF}_{e'}^N$. Now suppose that y is an optimal solution of $\mathcal{MCF}_{e'}^N$ with the additional constraint that flow on the root edge e' of T_e^N , denoted by $y_{e'}$, is fixed to value $z_e^* - 1$. Then, using arguments similar to those used in the proof of Theorem 4.1, it can be shown that there exists an augmenting path W of y with respect to z^* of length $2n^2 c_{\max}$ in $T_{e_0}^N$. As before, W can be decomposed into at least $2nc_{\max}$ disjoint simple cycles and a simple path. Now each cycle has a cost of at least $\delta(x^*)$, which is at least 1 because $\mathcal{MCF}$ has integral data. Because the $\mathcal{MCF}$

and $\mathcal{MCF}_e^N$ have integral parameters, the y and x^* can be restricted to be integral. Therefore, the augmenting path W must allow for pushing at least one unit amount of flow to modify y to result in the decrease of its cost by at least $nc_{\max}$. This is because (a) the increase, due to pushing a unit amount of flow on the simple path, could be at most $nc_{\max}$, and (b) decrease along (at least) $2nc_{\max}$ cycles is at least $2nc_{\max}$. In summary, the modified solution is feasible for $\mathcal{MCF}_e^N$ on T_e^N with cost decreased by at least $nc_{\max}$. Therefore, it would follow that the optimal cost $b_e^N(z_e^*)$ for $\mathcal{MCF}_e^N$ is less than $b_e^N(z_e^* - 1) - nc_{\max}$. In a very similar manner, it can be argued that $b_e^N(z_e^*) < b_e^N(z_e^* + 1) - nc_{\max}$. This concludes that $\min(b_e^N(z_e^* + 1), b_e^N(z_e^* - 1))$ is at least $b_e^N(z_e^*) + nc_{\max}$.

To establish the other side of the equivalence, suppose $\mathcal{MCF}$ does not have a unique optimal solution. Consider any arc $e \in E$, corresponding computation tree T_e^N , and optimization problem $\mathcal{MCF}_e^N$. Let e' be the root arc of T_e^N as before. Let y be the optimal assignment of $\mathcal{MCF}_e^N$ with the assignment for root arc e' being $y_{e'} = z_e^*$. Now, because $\mathcal{MCF}$ has multiple optimal solutions, there exists another optimal assignment x^* of $\mathcal{MCF}$ so that $x_e^* \neq z_e^*$. Indeed, given that both $\mathcal{MCF}_e^N$ and $\mathcal{MCF}$ are integral, we can restrict our attention to z_e^* , x^* , and y having integral components. Because $x_e^* \neq z_e^*$, using arguments similar to those used in the proof of Theorem 4.1, it is indeed possible to find an augmenting path W , of length $2N$, on T_e^N with respect to y and x^* . This augmenting path decomposes into one simple path P of length at most $n - 1$ and at least $2nc_{\max}$ simple cycles. Because x^* is an optimal solution, the cost of each of the cycles with respect to the residual graph $G(x^*)$ is nonpositive (it is not strictly negative like the proof of Theorem 4.1 because the x^* is not unique). The cost of the path, however, is between $-(n - 1)c_{\max}$ and $(n - 1)c_{\max}$. Therefore, by pushing the unit amount of flow (which is possible along this augmenting path W due to integrality of x^* and y), the resulting flow $\tilde{y}$ on T_e^N is such that its total cost is at most $(n - 1)c_{\max}$ more than the cost of y . Now either $\tilde{y}_{e'} = z_e^* - 1$ or $z_e^* + 1$. Suppose $\tilde{y}_{e'} = z_e^* - 1$. In that case, the $\tilde{y}$ is a feasible solution of $\mathcal{MCF}_e^N$ with the additional constraint that the root arc e' has flow $z_e^* - 1$. This cost is no less than the cost of an optimal solution of $\mathcal{MCF}_e^N$ with the additional constraint that the root arc e' has flow $z_e^* - 1$, which is defined as $b_e^N(z_e^* - 1)$. Putting all this together, we obtain

$$b_e^N(z_e^* - 1) \leq b_e^N(z_e^*) + nc_{\max}.$$

In a similar manner, if $\tilde{y}_{e'} = z_e^* + 1$, then we would conclude that

$$b_e^N(z_e^* + 1) \leq b_e^N(z_e^*) + nc_{\max}.$$

That is, we have established that if $\mathcal{MCF}$ does not have a unique optimal solution, then

$$\min(b_e^N(z_e^* - 1), b_e^N(z_e^* + 1)) \leq b_e^N(z_e^*) + nc_{\max}.$$

This completes the proof of the other side of equivalence, and hence the proof of Corollary 5.2. $\square$

6. Network Flow: Piecewise-Linear Convex Objective

This section describes the extension of Theorem 4.1 for the network flow problem with piecewise-linear convex objective or cost function. Specifically, given a graph $G = (V, E)$ as before, consider

$$\begin{aligned} & \text{minimize} \quad \sum_{e \in E} c_e(x_e) \\ & \text{subject to} \quad \sum_{e \in E_v} \Delta(v, e)x_e = f_v, \quad \forall v \in V \\ & \quad \quad \quad (\text{demand/supply constraints}) \\ & \quad \quad \quad 0 \leq x_e \leq u_e, \quad \forall e \in E \\ & \quad \quad \quad (\text{nonnegativity constraints}), \end{aligned} \quad (\mathcal{CP})$$

where $c_e: \mathbb{R} \rightarrow \mathbb{R}$ is a piecewise-linear convex function for each $e \in E$. As before, we shall assume that the $\mathcal{CP}$ is feasible. Let ψ be the same as before, and define

$$\phi_e(z) = \begin{cases} c_e(z) & \text{if } 0 \leq z \leq u_e \\ \infty & \text{otherwise.} \end{cases}$$

Algorithm 2 on G with functions ψ and ϕ thus defined is the BP for this problem instance. Before we state our result, we need to define the corresponding residual graph. Suppose x is a feasible solution for $\mathcal{CP}$. Define the residual graph of G and x , denoted by $G(x)$ as follows: $\forall e = (v_\alpha, v_\beta) \in E$, if $x_e < u_e$, then e is an arc in $G(x)$ with cost $c_e^x = \lim_{t \downarrow 0} ((c(x_e + t) - c(x_e))/t)$; if $x_e > 0$, then there is an arc $e' = (v_\beta, v_\alpha)$ in $G(x)$ with cost $c_{e'}^x = \lim_{t \downarrow 0} ((c(x_e) - c(x_e - t))/t)$. Finally, let

$$\delta(x) = \min_{\mathcal{C} \in \mathcal{C}} \left\{ \sum_{e \in \mathcal{C}} c_e^x \right\},$$

where $\mathcal{C}$ is the set of all directed simple cycles in $G(x)$. We state the result about the convergence property of BP.

THEOREM 6.1. *Suppose x^* is the unique optimal solution for $\mathcal{CP}$, and hence $\delta(x^*) > 0$. Let L be the maximum cost of a simple directed path in $G(x^*)$. Then, for any $N \geq \lfloor L/(2\delta(x^*)) \rfloor + 1$, $\hat{x}^N = x^*$.*

The proof of Theorem 6.1 is identical to that of Theorem 4.1 with the above-defined notions. Therefore, we shall skip it.

7. Integral $\mathcal{MCF}$: Run-Time Analysis of BP

In the next two sections, we shall consider $\mathcal{MCF}$ with integral components for c , u , and f . Our goal is to analyze the run time of BP for such integral $\mathcal{MCF}$.

LEMMA 7.1. *For an integral $\mathcal{MCF}$, the total number of operations performed by Algorithm 2 to update all the messages at iteration t is $O(tc_{\max} m \log n)$.*

PROOF. Recall that for edge $e \in E$ with v as one of its end points (and w at the other), the message function is

updated as

$$m'_{e \rightarrow v}(z) = \phi_e(z) + \min_{\bar{z} \in \mathbb{R}^{|E_w|}, \bar{z}_e = z} \left\{ \psi_w(\bar{z}) + \sum_{\bar{e} \in E_w \setminus e} m'_{\bar{e} \rightarrow w}(\bar{z}_{\bar{e}}) \right\}.$$

From Corollary 4.12, all of the message functions have integral slopes for an instance of $\mathcal{MCF}$ with integral components. The absolute values of these slopes are bounded by $(t-1)c_{\max}$. This implies that each (convex piecewise-linear) message (function) has at most $2(t-1)c_{\max}$ linear pieces. By Theorem 4.9 and Observation 4.10 it follows that $g(z)$ can be computed in $O(tc_{\max}|E_w|\log|E_w|) = O(tc_{\max}|E_w|\log n)$ total operations because $|E_w| \leq n$. Here,

$$g(z) = \min_{\bar{z} \in \mathbb{R}^{|E_w|}, \bar{z}_e = z} \left\{ \psi_w(\bar{z}) + \sum_{\bar{e} \in E_w \setminus e} m'_{\bar{e} \rightarrow w}(\bar{z}_{\bar{e}}) \right\}.$$

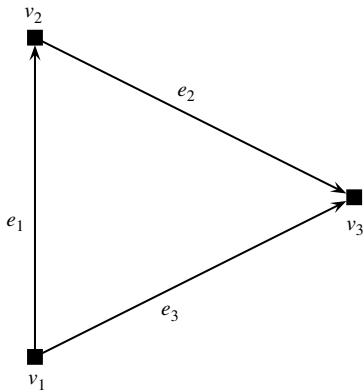
Now, computing $g(z) + \phi_e(z)$ is a simple procedure that requires increasing the slopes of linear pieces of $g(z)$ by a constant. Because $g(\cdot)$ has at most $2tc_{\max}$ linear pieces, computing $g(z) + \phi_e(z)$ takes further $O(tc_{\max})$ operations. In summary, it follows that all message updates can be performed in a total of $O(tc_{\max}m \log n)$ operations because $\sum_w |E_w| = \Theta(m)$. $\square$

We now complete the proof of Theorem 4.2.

PROOF OF THEOREM 4.2. The integral instance of $\mathcal{MCF}$ with a unique optimal solution has $\delta(x^*) \geq 1$. Therefore, by Theorem 4.1, the BP Algorithm 2 converges after at most $O(nL)$ iterations. By Lemma 7.1, the total computation performed up to iteration t is $O(m \log nc_{\max} t^2)$. Therefore, the total computation performed until convergence is $O(m \log nc_{\max} n^2 L^2)$. The L can be bounded as $L = O(nc_{\max})$. Therefore, it follows that the overall cost is at most $O(mn^4 c_{\max}^3 \log n)$. $\square$

The bound of Theorem 4.2 is pseudopolynomial time. In fact, qualitatively this is the best bound one can hope for. To see this, consider an example of $\mathcal{MCF}$ defined on a directed graph G as shown in Figure 6. Given large

Figure 6. An $\mathcal{MCF}$ instance with exponential running time.



integer D , set the costs of edges as $c_{e_1} = c_{e_2} = D$, $c_{e_3} = 2D - 1$; demands as $b_{v_1} = 1$, $b_{v_2} = 0$, and $b_{v_3} = -1$. It can be checked that $\hat{x}_1^N$ alternates between 1 and -1 when $2N + 1 < (2D)/3$. This means that the BP algorithm takes at least $\Omega(D)$ iterations to converge. Because the input size is $\Theta(\log D)$, we have that Algorithm 2 for $\mathcal{MCF}$ does not converge to the unique optimal solution in polynomial time in the size of the input.

7.1. Run Time of BP for Integral $\mathcal{MCF}^o$

Here we analyze the run time of BP for integral $\mathcal{MCF}^o$, the subclass of $\mathcal{MCF}$ defined in §4.2, and prove Theorem 4.14.

PROOF OF THEOREM 4.14. Because $\mathcal{MCF}^o$ is an instance of $\mathcal{MCF}$ with integral components and a unique optimal solution, from Theorem 4.1 it follows that the BP Algorithm 2 converges to the optimal solution within $O(Ln)$ iterations. To bound computation performed in each iteration and subsequently bound overall computation cost, without loss of generality we shall assume that the piecewise-linear convex message function is such that each linear piece is of unit length. This assumption is without loss of generality, because each linear piece has integral vertices from Corollary 4.13, and hence assumption of each piece being unit length only leads to upper bound on computation. Now each message function is defined on a uniformly bounded interval due to uniform bound K on the capacity of each edge in $\mathcal{MCF}^o$. Therefore, the number of pieces in each piecewise-linear convex message function is bounded by $K + 1$. Recall that for $t \geq 1$,

$$m'_{e \rightarrow v}(z) = \phi_e(z) + \min_{\bar{z} \in \mathbb{R}^{|E_w|}, \bar{z}_e = z} \left\{ \psi_w(\bar{z}) + \sum_{\bar{e} \in E_w \setminus e} m'_{\bar{e} \rightarrow w}(\bar{z}_{\bar{e}}) \right\}.$$

As explained in detail in §4.1, specifically Lemma 4.6 and Theorem 4.9, computing $m'_{e \rightarrow v}$ takes at most $O(K \log |E_w|)$, which is $O(K \log n)$ because $|E_w| \leq n$ for all w . Because there are at most $O(m)$ messages, total computation per iteration is $O(Km \log n)$. As discussed earlier, it takes $O(Ln)$ iterations for the algorithm to converge. Therefore, overall computation scales $O(KLmn \log n)$. Finally, due to a uniform bound of K on the cost of edges, $L = O(nc_{\max}) = O(nK)$. In summary, the total computation cost is bounded above by $O(K^2 mn^2 \log n)$. $\square$

8. FPRAS for $\mathcal{MCF}$ Using BP

In this section, we provide a fully polynomial-time randomized approximation scheme (FPRAS) for $\mathcal{MCF}$, using BP as a subroutine. As mentioned earlier, we shall assume integral $\mathcal{MCF}$. We start by describing the insights behind the algorithm, followed by precise description in §8.2. To this end, recall that the key hurdles in making BP fully polynomial time as indicated by Theorem 4.2 are the following:

1. The convergence of BP requires $\mathcal{MCF}$ to have a unique optimal solution.

2. The running time of BP is polynomial in m , n , and $c_{\max}$.

Therefore, to find FPRAS for any given instance of $\overline{\mathcal{MCF}}$, we need to overcome the requirement of uniqueness and dependence over $c_{\max}$ of running time. To do so, we shall utilize the appropriate randomized modification of the cost vector so that the resulting problem with modified cost vector $\bar{c}$ has the following properties:

1. The modified problem has a unique optimal solution with high probability.

2. The modified cost vector has $\bar{c}_{\max}$ polynomial in m , n , and $1/\varepsilon$.

3. The optimal solution of the modified problem provides a $1 + \varepsilon$ multiplicative approximation to the optimal solution of $\overline{\mathcal{MCF}}$.

It seems intuitive that by adding enough randomness to the cost vector, the modified problem will have a unique solution with high probability. However, requiring the resulting cost vector to be polynomially small in m , n , and $1/\varepsilon$ as well as having a small approximation error is challenging, and a priori not clear if it is even feasible. The so-called Isolation Lemma introduced in Mulmuley et al. (1987) helps to address precisely this question for a specific class of combinatorial problems, including matching. It is not directly applicable to our setup, primarily because the Isolation Lemma requires the feasible set of the optimization problem to be a monotone subset of $\{0, 1\}^M$ (for appropriate M), whereas the feasible set of interest here is a polytope derived from a linear programming problem. For this reason, we state and prove a variation of the Isolation Lemma for our setup next.

8.1. Variation of the Isolation Lemma

THEOREM 8.1. *Let $\overline{\mathcal{MCF}}$ be an instance of the min-cost flow problem with underlying graph $G = (V, E)$, demand vector b , constraint vector u . Let its cost vector $\bar{c}$ be generated as follows: for each $e \in E$, $\bar{c}_e$ is chosen independently and uniformly over N_e , where N_e is a discrete set of $4m$ positive numbers ($m = |E|$). Then, the probability that $\overline{\mathcal{MCF}}$ has a unique optimal solution is at least $\frac{1}{2}$.*

PROOF. Fix an arc $e_1 \in E$ and fix $\bar{c}_e$ for all $e \in E \setminus e_1$. First suppose that there exists a value $\alpha \geq 0$ such that when $\bar{c}_{e_1} = \alpha$, $\overline{\mathcal{MCF}}$ has two optimal solutions x^* , x^{**} and, moreover, $x_{e_1}^* = 0$ and $x_{e_1}^{**} > 0$. Then, if $\bar{c}_{e_1} > \alpha$, for any feasible solution x of $\overline{\mathcal{MCF}}$ with $x_{e_1} > 0$,

$$\begin{aligned} \sum_{e \in E} \bar{c}_e x_e^* &= \sum_{e \in E, e \neq e_1} \bar{c}_e x_e^* \\ &\stackrel{(a)}{\leq} \sum_{e \in E, e \neq e_1} \bar{c}_e x_e + x_{e_1} \alpha \\ &\stackrel{(b)}{<} \sum_{e \in E} \bar{c}_e x_e. \end{aligned}$$

In the above, (a) follows from the fact that x^* is optimal with $\bar{c}_{e_1} = \alpha$; (b) follows $\bar{c}_{e_1} > \alpha$ and $x_{e_1} > 0$. On the other

hand, if $\bar{c}_{e_1} < \alpha$, then for any feasible solution x of $\overline{\mathcal{MCF}}$ where $x_{e_1} = 0$, we have

$$\begin{aligned} \sum_{e \in E} \bar{c}_e x_e^{**} &\stackrel{(a)}{<} \sum_{e \in E, e \neq e_1} \bar{c}_e x_e^{**} + \alpha x_{e_1}^{**} \\ &\stackrel{(b)}{\leq} \sum_{e \in E, e \neq e_1} \bar{c}_e x_e + \alpha x_{e_1} \\ &= \sum_{e \in E} \bar{c}_e x_e. \end{aligned}$$

Above, (a) follows from $x_{e_1}^{**} > 0$ and $\bar{c}_{e_1} < \alpha$; (b) follows from x^{**} being an optimal solution with $\bar{c}_{e_1} = \alpha$. In summary, there exists at most one value for α such that when $\bar{c}_{e_1} = \alpha$, $\overline{\mathcal{MCF}}$ has two solutions x^* , x^{**} with $x_{e_1}^* = 0$ and $x_{e_1}^{**} > 0$. In a similar manner, it can be established that there exists at most one value β such that with $\bar{c}_{e_1} = \beta$, $\overline{\mathcal{MCF}}$ has two optimal solutions x^* , x^{**} with $x_{e_1}^* < u_{e_1}$ and $x_{e_1}^{**} = u_{e_1}$.

Let $\mathcal{O}$ be the set of all optimal solutions of $\overline{\mathcal{MCF}}$. From the above discussion, it follows that for a given arc e , if $\bar{c}_e$ is chosen uniformly at random from $4m$ distinct positive integers, then the probability that there exist two solutions x^* , x^{**} in $\mathcal{O}$ that satisfy either $x_e^* = 0$, $x_e^{**} > 0$ or $x_e^* < u_e$, $x_e^{**} = u_e$ is at most $1/(2m)$. Therefore, with probability at least $1 - 1/(2m)$, all solutions x in $\mathcal{O}$ satisfy either $x_e = 0$ or $0 < x_e < u_e$ or $x_e = u_e$. Denote this event by $D(e)$. By union, bound $\bigcap_{e \in E} D(e)$ holds with probability at least $1/2$. Now to conclude the proof of Theorem 8.1, we state the following lemma.

LEMMA 8.2. *Under event $\bigcap_{e \in E} D(e)$, the $\overline{\mathcal{MCF}}$ has a unique optimal solution.*

PROOF. Suppose to the contrary that under event $\bigcap_{e \in E} D(e)$, $\overline{\mathcal{MCF}}$ has two distinct optimal solutions x^* and x^{**} . Let $d = x^{**} - x^*$; then $x^* + \lambda d$ is an optimal solution of $\overline{\mathcal{MCF}}$ iff $0 \leq (x^* + \lambda d)_e \leq u_e$, $\forall e \in E$. Because $\bar{c}_e > 0$ for any $e \in E$ and $\bar{c}^T d = \bar{c}^T x^{**} - \bar{c}^T x^* = 0$, there exists some $e' \in E$ such that $d_{e'} < 0$. Let

$$\lambda^* = \sup\{\lambda \geq 0: x^* + \lambda d \text{ is a feasible solution of } \overline{\mathcal{MCF}}\}.$$

Because $d_{e'} < 0$, λ^* is bounded and because $x^* + d = x^{**}$, $\lambda^* \geq 1$. Further, the supremum λ^* is achieved, that is, $x^* + \lambda^* d$ is a feasible solution of $\overline{\mathcal{MCF}}$ because the feasible space of $\overline{\mathcal{MCF}}$ is a closed set. By definition of λ^* , there must exist some e'' such that $x_{e''}^* \neq x_{e''}^{**}$ and either $(x^* + \lambda^* d)_{e''} = 0$ or $u_{e''}$. Because $\lambda^* > 0$, $x_{e''}^* \neq (x^* + \lambda^* d)_{e''}$. That is, we have two solutions x^* and $x^* + \lambda^* d$ that do not satisfy $D(e'')$. This contradicts the hypothesis, and hence $\overline{\mathcal{MCF}}$ must have a unique optimal solution. $\square$

We note that Theorem 8.1 can be easily modified for LP in the standard form.

COROLLARY 8.3. *Let $\overline{\mathcal{LP}}$ be an LP problem with constraint $Ax = b$, where A is an $m \times n$ matrix, $b \in \mathbb{R}^m$. The cost vector $\bar{c}$ of $\overline{\mathcal{LP}}$ is generated as follows: for each $e \in E$, $\bar{c}_e$ is chosen independently and uniformly over N_e , where*

N_e is a discrete set of $2n$ elements. Then, the probability that $\overline{\mathcal{MCF}}$ has a unique optimal solution is at least $\frac{1}{2}$.

8.2. Finding the Correct Modified Cost Vector $\bar{c}$

Next, we construct a randomly generated cost vector $\bar{c}$ with the desired properties stated in the beginning of this section. Let $X: E \rightarrow \{1, 2, \dots, 4m\}$ be a random function where for each $e \in E$, $X(e)$ is chosen independently and uniformly over the range. Let $t = (c_{\max} \varepsilon)/(4mn)$ and generate $\bar{c}$ as follows: for each $e \in E$, let $\bar{c}_e = 4m \lfloor c_e/t \rfloor + X(e)$. Then, $\bar{c}_{\max}$ is polynomial in m , n , and $1/\varepsilon$. By Theorem 8.1, the probability of $\overline{\mathcal{MCF}}$ having a unique optimal solution is greater than $1/2$.

Now we introduce algorithm APRXMT($\overline{\mathcal{MCF}}, \varepsilon$) as follows. Select a random $\bar{c}$; try to solve $\overline{\mathcal{MCF}}$ using BP. If BP discovers that $\overline{\mathcal{MCF}}$ has no unique optimal solution (using Corollary 5.2), then restart the procedure by selecting another $\bar{c}$ at random; otherwise, return the unique optimal solution found by BP. Formally, we present APRXMT($\overline{\mathcal{MCF}}, \varepsilon$) as Algorithm 3.

Algorithm 3 APRXMT($\overline{\mathcal{MCF}}, \varepsilon$)

- 1: Let $t = (c_{\max} \varepsilon)/(4mn)$, for any $e \in E$, assign $\bar{c}_e = 4m \cdot \lfloor c_e/t \rfloor + p_e$, where p_e is an integer chosen independently, uniformly random from $\{1, 2, \dots, 4m\}$.
- 2: Let $\overline{\mathcal{MCF}}$ be the problem with modified cost $\bar{c}$.
- 3: Run Algorithm 2 on $\overline{\mathcal{MCF}}$ for $N = 2\bar{c}_{\max} n^2$ iterations.
- 4: Use Corollary 5.2 to determine if $\overline{\mathcal{MCF}}$ has a unique solution.
- 5: **if** $\overline{\mathcal{MCF}}$ does not have a unique solution **then**
- 6: Restart the procedure APRXMT($\overline{\mathcal{MCF}}, \varepsilon$).
- 7: **else**
- 8: Terminate and return $x^{(2)} = \hat{x}^N$, where $\hat{x}^N$ is the estimate of optimal flow assignments found in Algorithm 2.
- 9: **end if**

COROLLARY 8.4. *The APRXMT($\overline{\mathcal{MCF}}, \varepsilon$) runs in $O((n^8 m^7 \log n)/\varepsilon^3)$ expected time.*

PROOF. Theorem 8.1 implies that on average $O(1)$ instances of $\overline{\mathcal{MCF}}$ are required to be solved by the BP. Each such instance requires running Algorithm 2 for $O(n^2 \bar{c}_{\max})$ iterations. Therefore, the total cost scales as $O(\bar{c}_{\max}^3 mn^4 \log n)$ on average by Lemma 7.1. Because $\bar{c}_{\max} = O((m^2 n)/\varepsilon)$, it is bounded as $O(\varepsilon^{-3} m^7 n^7 \log n)$. $\square$

Now let $\bar{c}$ be the randomly chosen vector as per the above-described procedure such that $\overline{\mathcal{MCF}}$ has a unique optimal solution, say $x^{(2)}$. Next, we show that $x^{(2)}$ is a “near-optimal” solution of $\mathcal{MCF}$. To accomplish this, let

$e' = \arg \max c_e$, ties broken arbitrarily, and define a new optimization problem $\underline{\mathcal{MCF}}$ as follows:

$$\begin{aligned} & \text{minimize} \quad \sum_{e \in E} c_e x_e \\ & \text{subject to} \quad \sum_{e \in E_v} \Delta(v, e) x_e = b_v, \quad \forall v \in V \\ & \quad \quad \quad (\text{demand/supply constraints}) \quad (\underline{\mathcal{MCF}}) \\ & \quad \quad \quad x_{e'} = x_{e'}^{(2)} \\ & \quad \quad \quad 0 \leq x_e \leq u_e, \quad \forall e \in E \text{ (flow constraints).} \end{aligned}$$

LEMMA 8.5. *Suppose $x^{(3)}$ is an optimal solution for $(\underline{\mathcal{MCF}})$ and $x^{(1)}$ is an optimal solution of $\mathcal{MCF}$. Then*

$$c^T x^{(3)} - c^T x^{(1)} \leq |x_{e'}^{(2)} - x_{e'}^{(1)}| nt.$$

PROOF. Let $d = x^{(2)} - x^{(1)}$. Call $\gamma \in \{-1, 0, 1\}^{|E|}$ a *synchronous cycle vector* of d if for any $e \in E$, $\gamma_e = 1$ only if $d_e > 0$, $\gamma_e = -1$ only if $d_e < 0$, and the set $\{e \in E: \gamma_e = 1 \text{ or } \gamma_e = -1\}$ forms exactly one directed cycle in G . Now d is an integral vector of circulation (i.e., d sends 0 unit amount of flow to every vertex $v \in V$) because it is the difference of two feasible solutions of the same network flow problem. Therefore, d can be decomposed as $\sum_{\gamma \in \mathcal{H}'} \gamma = d$ with $\mathcal{H}' \subset \mathcal{H}$ and $\mathcal{H}$ being a finite set of synchronous cycle vectors of G (cf. see Ahuja et al. 1993). For any $\gamma \in \mathcal{H}'$, observe that $x^{(2)} - \gamma$ is a feasible solution for $\overline{\mathcal{MCF}}$. Now, because $x^{(2)}$ is an optimal solution for $\overline{\mathcal{MCF}}$, it follows that $\bar{c}^T \gamma \leq 0$. Now for any $e \in E$,

$$\begin{aligned} \bar{c}_e &= 4m \left\lfloor \frac{c_e}{t} \right\rfloor + p_e, \quad 1 \leq p_e \leq 4m, \\ &\Rightarrow \bar{c}_e, \frac{4mc_e}{t} \in \left[4m \left\lfloor \frac{c_e}{t} \right\rfloor, 4m \left(\left\lfloor \frac{c_e}{t} \right\rfloor + 1 \right) \right], \\ &\Rightarrow \left| \bar{c}_e - \frac{4mc_e}{t} \right| \leq 4m, \\ &\Rightarrow \sum_e \left| \frac{4mc_e}{t} - \bar{c}_e \right| |\gamma_e| \leq 4m \sum_e |\gamma_e| \leq 4mn. \end{aligned}$$

Using this and the fact that $\bar{c}^T \gamma \leq 0$, we have

$$\begin{aligned} \frac{4m}{t} c^T \gamma &\leq \frac{4m}{t} c^T \gamma - \bar{c}^T \gamma \\ &\leq \sum_e \left| \frac{4mc_e}{t} - \bar{c}_e \right| |\gamma_e| \\ &\leq 4mn. \end{aligned}$$

Therefore, we have $c^T \gamma \leq nt$. By definition of $\mathcal{H}'$, $x^{(2)} = x^{(1)} + \sum_{\gamma \in \mathcal{H}'} \gamma$. Therefore, for all $e \in E$

$$\min\{x_e^{(1)}, x_e^{(2)}\} \leq x_e^{(1)} + \sum_{\gamma \in \mathcal{H}'} \gamma_e \leq \max\{x_e^{(1)}, x_e^{(2)}\}.$$

Therefore, it follows that $x^{(1)} + \sum_{\gamma \in \mathcal{H}'} \gamma$ is a feasible solution for $\mathcal{MCF}$. Because $x^{(3)}$ is the optimal solution of $\mathcal{MCF}$,

$$\begin{aligned} c^T x^{(3)} &\leq c^T x^{(1)} + \sum_{\gamma \in \mathcal{H}'} c^T \gamma \\ &\leq c^T x^{(1)} + |\mathcal{H}'| nt. \end{aligned}$$

Because $|\mathcal{H}'| \leq |x_{e'}^{(2)} - x_{e'}^{(1)}|$, it follows that

$$c^T x^{(3)} - c^T x^{(1)} \leq |x_{e'}^{(2)} - x_{e'}^{(1)}| nt. \quad \square$$

COROLLARY 8.6. For any $\varepsilon \in (0, 1)$,

$$c^T x^{(3)} \leq \left(1 + \frac{\varepsilon}{2m}\right) c^T x^{(1)}.$$

PROOF. By Lemma 8.5 we may assume without loss of generality that $x_{e'}^{(2)} \neq x_{e'}^{(1)}$. Also by Lemma 8.5,

$$\begin{aligned} \frac{c^T x^{(3)} - c^T x^{(1)}}{c^T x^{(3)}} &\leq \frac{|x_{e'}^{(2)} - x_{e'}^{(1)}| nt}{c^T x^{(3)}} \\ &\leq \frac{|x_{e'}^{(2)} - x_{e'}^{(1)}| nt}{|x_{e'}^{(2)} - x_{e'}^{(1)}| c_{e'}} = \frac{nt}{c_{e'}}, \end{aligned} \quad (12)$$

where the last inequality follows because of $c^T x^{(3)} \geq |x_{e'}^{(2)} - x_{e'}^{(1)}| c_{e'}$ justified as follows: using $x_{e'}^{(3)} = x_{e'}^{(2)}$ by definition,

$$c^T x^{(3)} \geq x_{e'}^{(2)} c_{e'} \geq (x_{e'}^{(2)} - x_{e'}^{(1)}) c_{e'};$$

the optimal solution $x^{(3)}$ of $\mathcal{MCF}$ is a feasible solution for $\mathcal{MCF}$, $x^{(1)}$ is optimal solution for $\mathcal{MCF}$, and therefore

$$c^T x^{(3)} \geq c^T x^{(1)} \geq x_{e'}^{(1)} c_{e'} \geq (x_{e'}^{(2)} - x_{e'}^{(1)}) c_{e'}.$$

That is, $c^T x^{(3)} \geq |x_{e'}^{(2)} - x_{e'}^{(1)}| c_{e'}$.

Using $t = (c_{e'} \varepsilon) / (4mn)$, from (12) it follows that

$$\frac{c^T x^{(3)} - c^T x^{(1)}}{c^T x^{(3)}} \leq \frac{\varepsilon}{4m}.$$

Therefore,

$$c^T x^{(3)} \leq \left(1 - \frac{\varepsilon}{4m}\right)^{-1} c^T x^{(1)} \leq \left(1 + \frac{\varepsilon}{2m}\right) c^T x^{(1)},$$

where the last inequality holds because $\varepsilon \in (0, 1)$. $\square$

8.3. The FPRAS

Loosely speaking, Corollary 8.6 shows that $x^{(2)}$ at arc e' is “near optimal” because fixing the flow at arc e' to $x_{e'}^{(2)}$ helps us in finding a feasible solution of $\mathcal{MCF}$ that is close to optimal. This leads us to an approximation algorithm $AS(\mathcal{MCF}, \varepsilon)$ (Algorithm 4) below. This algorithm at every iteration uses APRXMT (Algorithm 3) and iteratively fixes the flow values at the arc with the largest cost. Theorem 8.7 establishes that this algorithm $AS(\mathcal{MCF}, \varepsilon)$ is indeed an FPRAS.

Algorithm 4 $AS(\mathcal{MCF}, \varepsilon)$

- 1: Let $G = (V, E)$ be the underlying directed graph of $\mathcal{MCF}$ with $m = |E|$, $n = |V|$.
- 2: **while** $\mathcal{MCF}$ flows for all arcs are not assigned **do**
- 3: Run APRXMT($\mathcal{MCF}, \varepsilon$), let $x^{(2)}$ be the solution returned.
- 4: Find $e' = \arg \max_{e \in E} c_e$ and modify $\mathcal{MCF}$ by fixing the flow on arc e' by $x_{e'}^{(2)}$; change the demands/supply on node v', w' with $e' = (v', w')$.
- 5: **end while**

THEOREM 8.7. Given $\varepsilon \in (0, 1)$, algorithm $AS(\mathcal{MCF}, \varepsilon)$ takes $O(\varepsilon^{-3} n^7 m^8 \log n)$ operations on average. Let x^* be the solution produced by $AS(\mathcal{MCF}, \varepsilon)$. Then

$$c^T x^* \leq (1 + \varepsilon) c^T x^{(1)}.$$

PROOF. By Corollary 8.4, APRXMT($\mathcal{MCF}, \varepsilon$) takes $O(\varepsilon^{-3} n^7 m^7 \log n)$ operations on average. Because $AS(\mathcal{MCF}, \varepsilon)$ invokes the method APRXMT($\mathcal{MCF}, \varepsilon$) m times, $AS(\mathcal{MCF}, \varepsilon)$ performs on average total operations bounded as $O(\varepsilon^{-3} n^7 m^8 \log n)$. By successive application of Corollary 8.6,

$$\begin{aligned} c^T x^* &\leq \left(1 + \frac{\varepsilon}{2m}\right)^m c^T x^{(1)} \\ &\leq e^{\varepsilon/2} c^T x^{(1)} \\ &\leq (1 + \varepsilon) c^T x^{(1)}, \end{aligned}$$

where the last two inequalities follows for $\varepsilon \in (0, 1)$ and $m \geq 1$. $\square$

9. Conclusions

In this paper, we formulated and analyzed the Belief Propagation (BP) algorithm for the capacitated min-cost network flow problem $\mathcal{MCF}$. We proved that the BP solves $\mathcal{MCF}$ exactly in pseudopolynomial time when the optimal solution is unique. This result generalizes an earlier result from Bayati et al. (2008a) and provides new insights for understanding BP as an optimization solver. Although the running time of BP for $\mathcal{MCF}$ is slower than other existing algorithms for $\mathcal{MCF}$, the advantage of BP is that it is a general-purpose distributed heuristic that is widely applicable and that is easy to formulate and implement for a broad class of constrained optimization problems. We also showed that a similar result holds for the network flow problem with the piecewise-linear convex cost function. A salient feature of the BP established in this work is the ability to detect the uniqueness of the optimal solution in an entirely distributed manner.

We showed that the BP algorithm, in its original form, at best leads to a pseudopolynomial-time algorithmic complexity. To address this problem, we have introduced a randomized variant of BP and showed that this variant

provides FPRAS. This is the first FPRAS result for the BP-type algorithms. Our variant of BP is based on fixing the values of flow variables one by one in a sequential manner. Such methodology, used commonly in practice, is known as the “decimation” procedure (see Montanari et al. 2007). To the best of our knowledge, this is the first disciplined, provable instance of the decimation procedure in the context of BP algorithms.

Acknowledgments

The authors thank the anonymous referees for the helpful comments. A conference version of this paper appeared in *Proceedings of the 21st ACM-SIAM Symposium on Discrete Algorithms* (Gamarnik et al. 2010). While working on this paper, D. Gamarnik was partially supported by the National Science Foundation [Project CMMI-0726733]; D. Shah was supported in part by the National Science Foundation [EMT Project CCF 0829893 and NSF CAREER project CNS 0546590]; and Y. Wei was partially supported by a Natural Sciences and Engineering Research Council of Canada Postgraduate Scholarship.

Endnote

1. A vertex v' is called leaf if it is connected to exactly one other vertex.

References

- Ahuja, R., A. Goldberg, J. Orlin, R. Tarjan. 1992. Finding minimum-cost flows by double scaling. *Math. Programming* **53** 243–266.
- Ahuja, R. K., T. L. Magnanti, J. B. Orlin. 1993. *Network Flows*. Prentice-Hall Inc, Englewood Cliffs, NJ.
- Aji, S. M., R. J. McEliece. 2000. The generalized distributive law. *IEEE Trans. Inform. Theory* **46**(2) 325–343.
- Bayati, M., D. Shah, M. Sharma. 2008a. Max-product for maximum weight matching: Convergence, correctness, and lp duality. *IEEE Trans. Inform. Theory* **54**(3) 1241–1251.
- Bayati, M., C. Borgs, J. Chayes, R. Zecchina. 2008b. On the exactness of the cavity method for weighted b -matchings on arbitrary graphs and its relation to linear programs. *J. Statist. Mech.: Theory and Experiment* L06001.
- Bertsekas, D. P. 1986. Distributed relaxation methods for linear network flow problems. *Proc. 25th IEEE Conf. Decision and Control*, Athens, Greece, 2101–2106.
- Bertsimas, D., J. Tsitsiklis. 1997. *Introduction to Linear Optimization*, 3rd ed. Athena Scientific, Belmont, MA, 289–290.
- Edmonds, J., R. M. Karp. 1972. Theoretical improvements in algorithmic efficiency for network flow problems. *J. ACM* **19**(2) 248–264.
- Fujishige, S. 1986. A capacity-rounding algorithm for the minimum-cost circulation problem: A dual framework of the Tardos algorithm. *Math. Programming* **35**(3) 298–308.
- Gallager, R. 1963. Low density parity check codes. Ph.D. thesis, Massachusetts Institute of Technology, Cambridge, MA.
- Gamarnik, D., D. Shah, Y. Wei. 2010. Belief propagation for min-cost network flow: Convergence and correctness. *Proc. 21st ACM-SIAM Sympos. Discrete Algorithms*, SIAM, Philadelphia, 279–292.
- Goldberg, A., R. Tarjan. 1987. Solving minimum-cost flow problems by successive approximation. *STOC '87: Proc. Nineteenth Annual ACM Sympos. Theory of Comput.*, ACM, New York, 7–18.
- Goldberg, A. V., R. E. Tarjan. 1989. Finding minimum-cost circulations by canceling negative. *J. ACM* **36**(4) 873–886.
- Horn, G. B. 1999. Iterative decoding and pseudocodewords. Ph.D. thesis, California Institute of Technology, Pasadena, CA.
- Kanoria, Y., M. Bayati, C. Borgs, J. T. Chayes, A. Montanari. 2011. Fast convergence of natural bargaining dynamics in exchange networks. *Proc. 22nd ACM-SIAM Sympos. Discrete Algorithms*, SIAM, Philadelphia, 1518–1537.
- Malioutov, D. M., J. K. Johnson, A. S. Willsky. 2006. Walk-sums and belief propagation in Gaussian graphical models. *J. Machine Learning Res.* **7** 2031–2064.
- Mezard, M., G. Parisi, R. Zecchina. 2002. Analytic and algorithmic solution of random satisfiability problems. *Science* **297** 812.
- Moallemi, C. C., B. Van Roy. 2007. Convergence of min-sum message passing for convex optimization. *45th Allerton Conf. Comm., Control, Comput.*, SIAM, Philadelphia.
- Moallemi, C. C., B. Van Roy. 2009. Convergence of the min-sum message passing for quadratic optimization. *IEEE Trans. Inform. Theory* **55**(5) 2413–2423.
- Montanari, A., F. Ricci-Tersenghi, G. Semerjian. 2007. Solving constraint satisfaction problems through belief propagation-guided decimation. *45th Allerton Conf. Comm., Control, Comput.*, SIAM, Philadelphia.
- Mulmuley, K., U. Vazirani, V. Vazirani. 1987. Matching is as easy as matrix inversion. *Combinatorica* **7**(1) 105–113.
- Orlin, J. B. 1993. A faster strongly polynomial minimum cost flow algorithm. *Oper. Res.* **41**(2) 338–350.
- Pearl, J. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Francisco.
- Richardson, T., R. Urbanke. 2001. The capacity of low-density parity check codes under message-passing decoding. *IEEE Trans. Inform. Theory* **47**(2) 599–618.
- Röck, H. 1980. Scaling techniques for minimal cost flow problems. *Discrete Structures Algorithms* 181–191.
- Ruozzi, N., S. Tatikonda. 2008. s - t paths using the min-sum algorithm. *Forty-Sixth Annual Allerton Conf. Comm., Control, Comput., Urbana, IL*, 918–921.
- Sanghavi, S., D. Malioutov, A. Willsky. 2007. Linear programming analysis of loopy belief propagation for weighted matching. *Proc. NIPS Conf., Vancouver, British Columbia, Canada*.
- Sanghavi, S., D. Shah, A. Willsky. 2009. Message-passing for maximum weight independent set. *IEEE Trans. Inform. Theory* **51**(11) 4822–4834.
- Schrijver, A. 2003. *Combinatorial Optimization*. Springer, Berlin.
- Tardos, E. 1985. A strongly polynomial minimum cost circulation algorithm. *Combinatorica* **5**(3) 247–255.
- Weiss, Y., W. Freeman. 2001. On the optimality of solutions of the max-product belief-propagation algorithm in arbitrary graphs. *IEEE Trans. Inform. Theory* **47**(2) 736–744.
- Yedidia, J., W. Freeman, Y. Weiss. 2002. Understanding belief propagation and its generalizations. Technical Report, TR-2001-22, Mitsubishi Electric Research Lab, Cambridge, MA.